TECHNICAL CASE STUDY

Khazana

An offline-first, end-to-end encrypted personal finance and wealth platform. No backend. No account. No telemetry. Shipped as two independent clients over one shared financial engine.

Flutter · Dart | Next.js 16 · React 19 | Drift + SQLCipher | Dexie · Web Crypto | Tauri 2

AES-256-GCM | Tailwind v4 · Zentra design system | iOS · Android · macOS · Web · Desktop

Sandhosh Sivan
Design, architecture, implementation, security model and release engineering

Solo build · 22 Jun – 12 Sep 2026

sandhoshsivan00@gmail.com

Contents

What is in this document

Written to be read three ways: as an engineering case study, as source material for a portfolio page, and as interview preparation. Every figure in it was counted from the repository, not estimated.

PART I — THE PRODUCT

01 Executive summary
The thirty-second and the two-minute answer to “what did you build?”

02 Problem and motivation
Why an offline finance app had to exist, and what the alternatives cost the user

03 What it does
The complete feature inventory across both clients

04 Every screen, and why it exists
All 57 routes, then twelve taken apart — purpose, means, reasoning and the cost accepted

05 The front end
The Zentra design system, the shell, the dashboard, and how one write reaches every screen for every period

PART II — HOW IT IS BUILT

06 System architecture
Two clients, four rings, one framework-free core

07 Architecture at four levels
Context, container, component and code — the C4 view of a system with almost no context

08 High-level and low-level design
The decomposition, then the four contracts precise enough to implement against

09 The model in UML
Entity relationships, class structure, the unlock sequence and the vault state machine

10 Data and persistence
23 encrypted tables, FTS5 search, six schema migrations, decimal money

11 Security and privacy engineering
Key lifecycle, threat model, and the decisions behind them

12 The financial engine
FIFO lots, XIRR, double-entry, capital-gains tax, scoring

13 Cross-platform parity
How two codebases are prevented from disagreeing about money

PART III — ENGINEERING PRACTICE

14 Decisions and trade-offs
The choices worth defending, and what was rejected

15 Testing, CI and measured performance
1,633 tests plus visual regression, three pipelines, benchmarks that are measured rather than claimed

16 Free for everyone
No Pro tier, no paywall, nothing to unlock

17 Metrics, limitations and roadmap
Including what does not work yet

APPENDICES — READY TO USE

A LinkedIn copy
Short post, long post, headline and About paragraph

B Website and portfolio copy
Hero, feature grid, tech list and a 150-word blurb

C Interview question bank
Sixty-three likely questions with prepared answers, plus seven STAR stories

D How this would be built again
The build order the repository actually followed, and what each phase had to prove

E Résumé kit
The link line, the stack line, bullets at three lengths, and the STAR stories cut to résumé scale

F The guided tour
A five-minute screen-by-screen walkthrough with the line to say at each stop

Section 01

Executive summary

~98,200	1,633	23	5	0
LINES HAND-WRITTEN	AUTOMATED TESTS	ENCRYPTED TABLES	PLATFORMS SHIPPED	SERVERS

The thirty-second version

Khazana is a personal finance and wealth app that keeps every rupee of your data encrypted on your own device. There is no backend, no account, and no telemetry — not as a feature, but as an architectural constraint I held for the entire build. It ships as two independently implemented clients, a Flutter app for iOS, Android and macOS and a Next.js web app that also packages as a desktop app, and both compute their numbers from the same financial engine written twice and kept honest by mirrored tests.

The two-minute version

Every personal finance app I looked at asked for something I was not willing to give: bank credentials, or a copy of my complete financial life on somebody else's server. So I built the version that does not ask. The user's PIN derives a 256-bit key through PBKDF2-HMAC-SHA256 at 600,000 iterations; on mobile that key opens a SQLCipher-encrypted SQLite file, on web it encrypts every record individually with AES-256-GCM. The key is never written to disk and is purged from memory on lock. There is no recovery path, which is a deliberate zero-knowledge trade-off rather than an oversight.

On top of that vault sits a genuine financial engine, not a spending chart. It does double-entry accounting where every journal entry sums to zero, lot-level portfolio tracking with FIFO matching, XIRR by numerical root-finding, an Indian capital-gains tax engine whose rules are versioned by disposal date, EMI amortisation with avalanche and snowball payoff simulation, insurance coverage-gap analysis, and a weighted financial-health score. All of it is pure functions with no I/O and no framework imports, which is what made it portable from Dart to TypeScript in the first place.

The part I would most want to talk through in an interview is not any one algorithm — it is the problem of shipping the same product twice without the two copies drifting apart. The answer was to make disagreement a test failure: byte-identical JSON fixtures loaded by both the Dart and the TypeScript suites, a shared specification file that drives both clients' feature gates, and cross-language assertions that pin the same inputs to the same outputs on both sides.

> **STATUS, STATED PLAINLY**
>
> Khazana is complete enough to use daily and is being prepared for the App Store, Play Store and direct desktop download. It is in **private beta with ten users** and has not been publicly released — the critical path is Google Play's twelve-tester, fourteen-day closed test for individual publishers, not code. The repository carries a candid twelve-item defect log, and this document reproduces the open entries rather than hiding them.

Section 02

Problem and motivation

I built Khazana because I wanted to track my own money properly and could not find a tool whose price I was willing to pay. The price was rarely money.

What the alternatives cost

APPROACH	WHAT IT ASKS FOR	WHY I REJECTED IT
Aggregator apps	Net-banking credentials or account-aggregator consent	Handing a third party read access to every account, permanently, to avoid typing. The blast radius of their breach becomes mine.
Cloud-sync trackers	A full copy of the ledger on their servers	My complete financial life, legible to their staff, their subpoenas and their next acquirer.
Spreadsheets	Nothing — but they give nothing back	No lot tracking, no XIRR, no tax view, no reminders. Every derived number is hand-maintained and therefore wrong within a month.
Wealth platforms	Assets under management	The tool's advice and my interest are structurally misaligned.

The constraint I set, and kept

One rule shaped every subsequent decision: **no server, ever**. Not a lightweight one, not "just for sync", not "just to validate licences". Once a server exists, so does an account; so does a database of other people's finances; so does a breach I have to disclose. Refusing it forced better engineering everywhere else — deterministic merge instead of a sync service, offline signature verification instead of a licensing endpoint, and a threat model I can state in one sentence.

Every architectural decision in this project is downstream of one refusal: there is no server, so there is nothing to breach, nothing to subpoena, and nothing to shut down when I stop paying for it.

What that constraint bought

- **A threat model small enough to reason about.** No JWT, OAuth, RBAC, CORS or CSRF surface exists to get wrong, because no endpoint exists. The entire security question reduces to "is on-device data unreadable without the PIN?"
- **Zero running cost.** The app has no marginal cost per user, which is what lets it be free for everyone without ads or a subscription.
- **Genuine offline capability.** Not a cached degraded mode — the device holds the truth, so a flight or a dead connection changes nothing.
- **Longevity.** If I stop maintaining it tomorrow, every installed copy keeps working. Nothing expires when a server is switched off.

Why two clients rather than one

Flutter reaches iOS, Android and macOS from one codebase, but Flutter-on-web produces a canvas application that is heavy to load and awkward to make properly accessible. A separate Next.js app renders real DOM, exports as fully static files, installs as a PWA and wraps in Tauri for Windows, macOS and Linux desktop builds. Rather than treat the second client as a compromise, I treated it as a design constraint that forced the financial logic out of the UI layer entirely — which is the single decision this codebase most benefits from.

Section 03

What it does

Twenty-six screens in the Flutter client and thirty routes in the web client, grouped into five areas. Every feature is free for everyone.

Money — the ledger

FEATURE	WHAT IT DOES
Transactions	Income, expense and investment entries with categories, merchants, notes and receipt attachments. Every entry writes a balanced pair of double-entry postings behind the scenes.
Accounts	A real chart of accounts with balances and transfers. Transfers debit the destination and credit the source, so they cancel out of net worth instead of being counted twice.
Quick add	Natural-language entry — "spent 450 on groceries at bigbasket yesterday" parses to amount, category, merchant and date.
Search	Full-text search over the whole ledger via an SQLite FTS5 virtual table with a porter tokenizer, benchmarked at roughly 0.13 ms across 10,000 transactions.
Calendar ledger	A month grid with per-day income, expense and net, plus budget indicators.
Budgets	Per-category limits with a configurable alert threshold and single-month rollover. Spend is always recomputed, never stored, so it cannot drift out of sync with the ledger.
Recurring	Rules that materialise missed occurrences on unlock, bounded so a long-stale rule cannot flood the ledger.
Goals	Targets with contributions, progress rings and a required-monthly-contribution calculation.

Invest — the portfolio

FEATURE	WHAT IT DOES
Holdings and lots	Lot-level positions across fourteen asset types — equity and ETFs, equity and debt mutual funds, gold, bonds, cash, real estate, crypto, fixed deposits, PPF/EPF, NPS, Sukanya Samriddhi, sovereign gold bonds and ULIPs.
Allocation	Donut and sunburst breakdowns by asset group, sector, industry, market cap and currency.
Sector P&L	Realised and unrealised profit and loss rolled up along any dimension, computed from FIFO lot matching.
XIRR	Money-weighted return across the whole portfolio or a single instrument.
Analytics	Concentration risk, return distribution histogram and drill-down.
Dividends	Dividend, bonus, buyback and interest events, detected from statement narrations and matched only against instruments actually held.
Tax centre	Short- and long-term capital-gains estimation under Indian rules, versioned by disposal date, with the ₹1.25 lakh annual exemption pooled and allocated deterministically.
Markets, watchlist, news	Opt-in index levels, a market clock, a watchlist and a news view. Market data fetches carry no personal data.

Protect — the downside

FEATURE	WHAT IT DOES
Liabilities	Loans and credit cards with EMI amortisation, utilisation, and an avalanche-versus-snowball payoff simulator that detects when a budget cannot cover accruing interest.
Insurance	Policies with premium tracking and coverage-gap analysis — life at roughly ten times annual income, health at the greater of ₹5 lakh or half of income.
Safety Net	A single 0–100 readiness score over four weighted pillars: emergency fund 35, life cover 25, health cover 25, retirement assets 15.

Overview — the answers

FEATURE	WHAT IT DOES
Dashboard	Net worth, key metrics, safe-to-spend, a generated narrative and a net-worth sparkline drawn from real daily snapshots.
Financial health	A 0–100 score across four weighted categories — Wealth 30, Protection 25, Efficiency 25, Future 20 — with the methodology shown to the user rather than hidden.
Reports	Income and expense columns, category donut, net-worth trend, over selectable windows.
Forecast	Cash-flow projection from the median of completed months, net-worth projection, and what-if debt scenarios.
Insights	Spending anomalies against a trailing three-month per-category average, required SIP for a goal, and safe-to-spend as the minimum of two honest ceilings. Pure rules — no LLM, no network.

Tools and data

FEATURE	WHAT IT DOES
Bank import	One auto-detecting CSV and XLSX engine rather than a parser per bank — column aliasing plus header scanning covers HDFC, SBI, ICICI, IDFC, Kotak and Qatari banks, handling both separate debit/credit columns and single signed amounts.
Broker import	Trade and lot import with charges broken out into brokerage, STT, stamp duty, GST and other, flagging rows for review when a trade date is missing — because that date decides short versus long term.
Deduplication	Fingerprinting on date, amount, direction and normalised narration, deduping within the file as well as against the existing vault. Every import is one undoable batch.
Auto-capture	Android notification-listener capture of bank alerts. The parser extracts amount, merchant and timestamp, discards the raw message immediately, and queues a draft — nothing auto-commits to the ledger.
Reconcile	Statement-balance reconciliation with an explicit uncleared list as the explanation for any difference.
Diagnostics	Self-checks for undecryptable records, journal entries whose postings do not sum to zero, postings against deleted accounts, unclean money strings, lot-to-holding mismatches and stale FX rates.
Reminders	Local notifications for bills, budgets, renewals, goals and a digest — with quiet hours, cooldowns, and amounts hidden by default.
Backup and restore	An encrypted, authenticated, versioned backup file, plus a plain CSV dump and an erase-everything command. All free forever.
Vaults and profiles	Multiple vaults, each with its own PIN and encryption key, and profiles inside a vault for self, spouse or business.
Ghost mode	One toggle masks every monetary value on screen for use in public.

Section 04

Every screen, and why it exists

Fifty-seven routes across two clients. This section lists all of them, then takes twelve apart — because a screen list proves surface area, and only the reasoning proves engineering.

57	31	26	0
ROUTES IN TOTAL	NEXT.JS	FLUTTER	BEHIND A PAYWALL

How the navigation is organised

The web client groups its routes into five sections — Overview, Invest, Money, Protect and Tools — declared once in `navConfig.ts` and consumed by the sidebar, the mobile drawer and the command palette alike. The Flutter client carries two shells chosen by width: on phones, four sections — Home, Money, Invest and Health — around a centre add action, each with a segmented sub-navigation for its sibling screens; on desktop, a persistent sidebar. The web client matches it with a floating four-tab dock and a raised + below 900px.

The two clients are not allowed to drift apart on this. A parity test asserts that the web bottom navigation lists the same four destinations as the Flutter tab bar. Navigation is the part of a product users build muscle memory for, and two clients that disagree about where Budget lives are two different products wearing one name.

One detail in the Flutter router is worth pulling out, because it began as a bug. The app had five tabs and twenty-seven screens, so most locations were not a tab. The shell originally resolved "which tab is selected" with a `startsWith` scan that fell back to index zero — which meant Budget, Goals, Liabilities and eight other screens all highlighted Dashboard as the current tab. The fix was an explicit map, because the relationship it encodes ("Reports lives under Score") is a product decision and not something a string prefix can be asked to infer.

The web client — 31 routes

ROUTE	FILE	PURPOSE
`/`	`page.tsx`	Client-side replace to `/dashboard`. Deliberately not a server `redirect()`, which compiles to a marker that breaks the Tauri window under static export.
`/dashboard`	`dashboard/page.tsx`	28 widgets the user can drag, resize and hide: balance hero, net worth, a chart board with a different chart on every card, portfolio, health score, insights, upcoming bills. Layout saved per profile.
`/score`	`score/page.tsx`	Financial-health gauge, method explainer, four category cards, score history.
`/reports`	`reports/page.tsx`	Income vs expense, balance sheet, category mix, month-by-month savings rate. Formatted Excel and PDF export alongside the raw CSV.
`/forecast`	`forecast/page.tsx`	Safe-to-spend with its workings shown, six-month cash flow, five-year net worth, debt what-if.
`/analytics`	`analytics/page.tsx`	Risk, diversification, return distribution, concentration, sunburst allocation.
`/investments`	`investments/page.tsx`	Portfolio overview: one asset-group filter driving six KPIs, four allocation donuts, gainers and losers.
`/holdings`	`holdings/page.tsx`	The full position grid with inline editor and import panel.
`/watchlist`	`watchlist/page.tsx`	Instruments tracked but not owned, with target gaps.
`/markets`	`markets/page.tsx`	Index levels and session state. Levels are synthesised; the market clock is real.
`/dividends`	`dividends/page.tsx`	Received vs expected income, yield on actual portfolio value, twelve-month history.

ROUTE	FILE	PURPOSE
/tax	tax/page.tsx	Indian capital-gains estimate under rules versioned by disposal date.
/accounts	accounts/page.tsx	Bank and cash accounts with balances derived from postings; archive rather than delete.
/transactions	transactions/page.tsx	One timeline over two entities, with search, four filters and free CSV export of the filtered set.
/add	add/page.tsx	Add or edit expense, income, investment or transfer, with a natural-language quick-add bar.
/calendar	calendar/page.tsx	Monday-first month grid showing income and expense per day, never a net figure.
/budget	budget/page.tsx	Envelope budgets with opt-in rollover, 50/30/20 card, per-category alert thresholds.
/recurring	recurring/page.tsx	Scheduled rules led by annual committed cost, with batch run, single post-now and editing a rule in place.
/goals	goals/page.tsx	Savings goals with progress rings and on-track hints.
/liabilities	liabilities/page.tsx	Cards and loans, EMI, utilisation, avalanche-versus-snowball payoff simulator.
/insurance	insurance/page.tsx	Policies with coverage-gap analysis against income-multiple guidelines.
/safety-net	safety-net/page.tsx	One readiness ring over emergency fund, life cover, health cover and safe assets.
/import	import/page.tsx	Assets and transactions, each parse → preview → confirm, with per-run undo.
/reconcile	reconcile/page.tsx	Tick entries against a statement; suggests entries matching the difference exactly.
/alerts	alerts/page.tsx	User rules on price, weight, budget and renewal, sharing one evaluator with the bell.
/diagnostics	diagnostics/page.tsx	Eleven integrity checks over the vault. Read-only by design.
/news	news/page.tsx	Synthesised headlines filtered to held and watched symbols, badged as such.
/help	help/page.tsx	FAQ, keyboard shortcuts, privacy chips, tour replay.
/privacy	privacy/page.tsx	The store-listing privacy policy, written to be true of the mobile app too.
/terms	terms/page.tsx	Terms, including an explicit not-financial-advice clause.
/settings	settings/page.tsx	Appearance (theme and accent), profiles with photos, notifications, storage durability, backup and restore, currency, feedback, wipe.

Not routes, but screens all the same: the vault gate, the command palette, the fifteen-step tour and the confirmation dialog are rendered as overlays above the router, because none of them is a place a user should be able to link to or reload into.

The Flutter client — 27 routes

ROUTE	SCREEN	PURPOSE
/unlock	UnlockGateScreen	Setup, unlock, biometrics and the failure cooldown. The only route reachable while locked.
/app/dashboard	DashboardScreen	Net-worth hero, stat tiles, trend, recent activity, health, insights, goals, allocation.
/app/transactions	TransactionsScreen	The ledger, flattened to one list so scrolling stays constant-cost.
/app/transactions/add	AddTransactionScreen	Validated entry form with natural-language quick add.
/app/search	SearchScreen	Autofocused field over the FTS5 index.
/app/budget	BudgetScreen	Envelope progress per category.
/app/calendar	CalendarLedgerScreen	Adaptive month ledger: one column on phones, three panels above 600px.
/app/accounts	AccountsScreen	Grouped balances from postings, plus liability cards.
/app/recurring	RecurringScreen	Scheduled rules and their next run.

ROUTE	SCREEN	PURPOSE
`/app/capture`	`CaptureInboxScreen`	Review queue for transactions parsed from SMS and notifications. Nothing auto-commits.
`/app/investments`	`InvestmentsScreen`	Portfolio stats, sunburst allocation, sector P&L, movers, per-position detail.
`/app/investments/breakdown`	`PortfolioBreakdownScreen`	Roll-up along a chosen dimension: sector, industry, market cap or asset class.
`/app/investments/analytics`	`AnalyticsScreen`	Risk gauges, return distribution and concentration — same formulas as the web twin.
`/app/investments/add-lot`	`AddLotScreen`	Buy or sell lot entry, with charges folded into cost basis.
`/app/investments/import-lots`	`ImportLotsScreen`	CSV lot import; rejected rows listed with reason and line number.
`/app/liabilities`	`LiabilitiesScreen`	Debt totals and payoff calculation.
`/app/insurance`	`InsuranceScreen`	Cover, premium and coverage-gap advice.
`/app/safety-net`	`SafetyNetScreen`	Readiness hero, in parity with the web route of the same name.
`/app/goals`	`GoalsScreen`	Goal cards and contributions.
`/app/reports`	`ReportsScreen`	Window selector, income and expense bars, category donut, net-worth trend.
`/app/score`	`ScoreScreen`	Health gauge, method card, weekly report, category detail disclosed in place.
`/app/import/bank`	`BankImportScreen`	Bank picker, staged preview, commit, and a persistent import report.
`/app/settings`	`SettingsScreen`	Vault, lock, tour, capture, currency, backup, restore, erase.
`/app/settings/market-data`	`MarketDataSettingsScreen`	Fallback price-provider keys, stored in the OS keychain.
`/app/settings/currency`	`CurrencySettingsScreen`	Manual exchange rates and their history.
`/app/settings/notifications`	`NotificationSettingsScreen`	Per-category toggles and the pending count read back from the OS scheduler.

Twelve screens worth defending

These twelve were not chosen for visual weight. Each one carries a decision that could reasonably have gone the other way, and the interesting part of each is the cost that was accepted rather than the feature that was delivered.

Unlock gate `/unlock · webapp VaultGate.tsx`

PURPOSE The only screen reachable while the vault is locked. Creates a vault, unlocks one, or refuses.

BUILT WITH PBKDF2-HMAC-SHA256 at 600,000 iterations over a 32-byte salt, via Web Crypto on the web and an off-isolate derivation on native. A router redirect guard holds every other route until the vault state machine says unlocked.

WHY The encryption key is never stored. Correctness is checked by decrypting a domain-separated verifier, so a wrong PIN fails on the authentication tag rather than against a stored secret. Biometric key storage exists but is opt-in, and the option is worded as its cost rather than its convenience.

> **Trade-off:** every unlock pays roughly 49 ms of key stretching, and there is no recovery path whatsoever — a forgotten PIN is a destroyed vault. Both are accepted deliberately: the first buys brute-force resistance, and the second is the honest consequence of having nobody who could perform a recovery.

Dashboard `/dashboard · dashboard/page.tsx`

PURPOSE The one screen that has to answer "how am I doing?" before the user scrolls.

BUILT WITH Thirteen store slices, memoised roll-ups, a month-pinning control, a narrative engine that composes sentences from a template table, and a widget grid the user arranges — described in Section 05.

WHY Net worth over time prefers recorded snapshots, but a new vault has none. Rather than show an empty chart, it falls back to a series derived by walking the transaction history — and labels that series *estimated*, because a derived line and a recorded line are different claims about the past.

> **Trade-off:** two code paths for one chart, and a label most users will not read. The alternative — silently presenting derived history as recorded history — is the kind of small dishonesty that makes every other number on the screen suspect once discovered.

Transactions `/transactions · TransactionsScreen`

PURPOSE One chronological timeline over two different entities — transactions and transfers.

BUILT WITH On Flutter, a `sealed class _Row` that flattens day headers and entries into a single list so the whole ledger stays one `ListView.builder`. On the web, fifty-row paging with search across merchant, note, category, account, formatted date and amount in any separator form.

WHY The obvious structure — a list of days, each containing a list of entries — builds every entry of every day eagerly. Flattening keeps scrolling constant-cost regardless of history length.

> **Trade-off:** the flattened model makes day-level operations harder to express, and the web client pages rather than virtualises. Pagination is recorded as the clearest remaining performance gap rather than presented as a design choice.

Add / edit entry `/add · add/page.tsx`

PURPOSE Every way money enters the ledger: expense, income, investment and transfer.

BUILT WITH A natural-language quick-add bar, an icon category grid, receipt attachment with type and size validation, and a confirmation step that names the amount, date and account deltas before saving an edit.

WHY A transfer is written as a *separate entity*, not as a pair of transactions. Moving money between your own accounts is not spending, and modelling it as spending would inflate every budget, expense total and savings rate that reads the transaction table.

> **Trade-off:** every screen that shows a timeline now has to merge two entity types instead of querying one. That cost is paid once, in the query layer, rather than repeatedly and invisibly in every aggregate.

Holdings `/holdings · holdings/page.tsx`

PURPOSE The position grid — every instrument owned, with cost, value and weight.

BUILT WITH A fourteen-column data grid with a locked first column, optional columns, weight micro-bars and CSV or XLSX export; inline editors that scroll themselves into view, honouring reduced-motion preferences.

WHY Fixed-income rows show rate, accrued interest and maturity instead of a price. A fixed deposit has no market price, and rendering one would be an invention; the accrual is computable and true.

> **Trade-off:** the grid now has two row shapes and cannot sort meaningfully on a single "current price" column across both. Sorting was the thing given up, and the alternative was fabricating a number in a financial application.

Portfolio overview `/investments · investments/page.tsx`

PURPOSE The portfolio answered at roll-up level rather than position level.

BUILT WITH One asset-group filter that drives every panel on the page, an "as on" date bounded by snapshot history, six KPIs, four allocation donuts and a diversification gauge computed from group count, sector spread, concentration and maximum weight.

WHY Three honesty banners sit above the numbers: unpriced positions counted at cost, exchange rates still at their built-in defaults, and a filtered or past-dated view. Each one exists because the total below it is understated or conditional, and the screen says so rather than letting the figure speak with unearned confidence.

> **Trade-off:** a screen that qualifies itself three times reads as less authoritative than one that does not. It is also the only one of the two that is correct.

Tax centre `/tax · tax/page.tsx`

PURPOSE An estimate of Indian capital-gains liability across the portfolio.

BUILT WITH A rule set versioned by effective date, FIFO lot matching, and a full methodology footer stating what was assumed.

WHY Gains are priced as *one portfolio* rather than per position, because the long-term exemption is an annual allowance applied once across all disposals — computing it per holding would multiply the exemption by the number of positions. Holdings with no recorded purchase date are excluded rather than defaulted to short-term, and slab-rate instruments return null with a banner instead of a guess.

> **Trade-off:** the screen shows fewer rows than the user owns and refuses to total some of them. A tax figure that is confidently wrong is worse than one that is visibly incomplete, because only the second can be corrected before it is filed.

Forecast `/forecast · forecast/page.tsx`

PURPOSE Safe-to-spend, six-month cash flow, five-year net worth and a debt what-if.

BUILT WITH Recurring rules for committed outflow, median rather than mean discretionary spend, and loop bounds on every projection so a corrupt rule cannot hang the page.

WHY Safe-to-spend is the minimum of two independent ceilings — cash after committed outgoings, and remaining budget headroom. A figure that ignored either would tell someone to spend money already owed, or money they do not have. When there is no income history or no spending history, it returns null with a typed reason rather than a number.

> **Trade-off:** new users see an explanation where they expected a figure. The screen shows its arithmetic line by line for exactly this reason: a forecast nobody can audit is a horoscope.

Import `/import · import/page.tsx`

PURPOSE Bank statements and broker exports into the ledger.

BUILT WITH One auto-detecting parser rather than a parser per bank: the delimiter is sniffed by column-count consistency, dates are day-first detected, and Excel serials are converted. Institution tiles select *guidance*, not code paths.

WHY Every run is parse → preview → confirm, written as a single batch, and recorded as an import batch that can be undone as a unit. Skipped rows are counted and shown; an import that silently drops rows is worse than one that fails.

> **Trade-off:** the undo restores what the import added but cannot restore prior values it never captured, and the interface says so rather than implying a general-purpose time machine.

Reconcile `/reconcile · reconcile/page.tsx`

PURPOSE Agree the ledger against a real bank statement.

BUILT WITH The classic arithmetic, stated rather than hidden: cleared balance is the opening balance plus every cleared entry up to the statement date, and the difference is the statement's closing balance minus that.

WHY When a difference exists, the screen searches for entries whose amounts match the gap exactly and offers them. That turns the hardest part of reconciliation — finding which entry is wrong — from a manual scan into a suggestion.

> **Trade-off:** the suggestion can be a coincidence, so it is offered rather than applied. An empty cleared-date field means "no upper bound", never "today", which is less convenient and avoids silently clearing future-dated entries.

Diagnostics `/diagnostics · diagnostics/page.tsx`

PURPOSE Eleven integrity checks over the vault: decryptability, ledger balance, posting coverage, orphaned accounts, balance-sheet balance, money-value cleanliness, attachment links, purchase dates, lot reconciliation and exchange-rate presence and staleness.

BUILT WITH Pure functions returning ok, warn or error with the offending record identifiers, and a stable check id that maps each failure to the screen that fixes it.

WHY **Nothing on this screen writes.** A diagnostic that repairs silently destroys the evidence of how the corruption happened, and the second occurrence is then unexplainable. Restore is gated on an explicit allowlist of four structural checks rather than on severity, because a restore blocked by an exchange rate nobody had entered yet is a file the user cannot get back into.

> **Trade-off:** the user has to act on what they are told instead of pressing one button. The alternative trades a debuggable system for a convenient one.

Settings `/settings · settings/page.tsx`

PURPOSE The largest screen in the product: appearance, profiles, notifications, privacy, storage, backup and restore, currency, sample data, lock and wipe.

BUILT WITH Two external-store subscriptions for notification preferences and browser permission state, so the toggle reflects the platform rather than the app's belief about it.

WHY Three details earn their place. Permission is requested on a click and "on" is never recorded if the browser says no. Storage durability is surfaced as a row with a retry, because Safari evicts script-writable storage after roughly seven days of no visits and a finance app silently losing its database is unacceptable. And export and restore take *separate* passphrases, so a backup can be handed over without handing over the vault PIN.

> **Trade-off:** two passphrases to remember instead of one, and a settings screen long enough to need its own navigation. Both are the cost of a local-first app having to expose the platform realities a server would otherwise absorb.

Section 05

The front end — design system, screens and live data

How the product looks, how it is put together on screen, and the part most finance apps get quietly wrong: making sure a number written in one place is the same number everywhere else, for whatever period the screen is showing.

1	2	18	28	1
TOKEN FILE	THEMES, ONE GEOMETRY	HAND-BUILT CHART TYPES	DASHBOARD WIDGETS	STORE, ONE RELOAD

Zentra — the visual language

The current look is called Zentra. Its rule fits on one line and is written at the top of the colour tokens: **ink is interaction, gold is identity, blue is data.** Chrome is neutral grey — canvas `#F2F2F2`, cards `#F8F8F8` lifted by a white inset top edge and a soft shadow rather than a border. Every button, active tab and active nav item is a near-black pill that turns white in dark mode. Gold appears on the brand mark and nowhere else. Colour is reserved for numbers: blue, green and pink series, green for income, and *pink rather than red* for expense, because spending is normal and red reads as an error.

All of it lives in one file, `webapp/src/app/globals.css`, as CSS custom properties in three blocks: the light theme on `:root`, the graphite dark theme on `:root[data-theme="dark"]`, and a theme-invariant block for radius (8 / 12 / 24 / 28 / pill), spacing (4 to 48 on a fixed scale), control heights, icon sizes, the type scale and motion durations. The invariant block encodes a rule worth stating in an interview: **switching theme changes colour and nothing else**, so no layout can shift when someone toggles dark mode. Tailwind v4's `@theme inline` maps the variables to utilities, so components write `bg-card text-ink-soft` and never a hex value.

CONCERN	HOW IT IS DONE, AND WHY
No flash of the wrong theme	An inline script in `layout.tsx` sets `data-theme` on `<html>` before first paint, reading the saved choice or following `prefers-color-scheme` for "system". After hydration the store owns it through `applyAppearance()`, and charts read the applied theme from `useDarkMode`, a `MutationObserver` on that attribute.
Accent colours	Six gemstone accents — Jade, Sapphire, Amethyst, Copper, Garnet, Lime — each a triple of accent, deep and glow per theme, applied as inline variables on `<html>`. The no-flash script carries its own copy of the table, so a test extracts it from `layout.tsx` by regex and asserts it equals the store's `ACCENTS`. A one-time migration reset pre-Zentra accents exactly once, behind a flag key.
Typography	Anek Latin in both clients, with tabular figures switched on globally so columns of money line up digit for digit. The `latin-ext` subset is loaded because the ₹ glyph lives there — without it the currency sign falls back to a different face mid-number.
Motion	Three durations and one easing curve as tokens. Reduced motion is honoured in three layers: a global CSS rule, Framer Motion's `reducedMotion="user"`, and the chart entrance hook that skips its draw-in.
Contrast, tested	Muted text moved from `#8E8E93` to `#6E6E73` to clear 4.5:1 on cards. Chart series are checked for 3:1 against the surface and for separation under simulated deuteranopia and protanopia. These are assertions in `palette.test.ts` with a Dart twin, not comments.

The shell

One breakpoint carries the layout: **900px**, deliberately not Tailwind's 768px `md`, because a sidebar plus a readable content column needs the extra room. Above it, a sticky 248px sidebar that collapses to a 76px icon rail (remembered per device); below it, a drawer for the full route index and a floating 64px dock with four tabs split two and two around a raised + that opens Add. The routes themselves are declared once in `navConfig.ts` as five groups and consumed by the sidebar, the drawer and the ⌘K command palette, so a new screen cannot appear in one and be missing from another. The four dock tabs match the Flutter tab bar, asserted by a test.

The dashboard as a product surface

A different chart on every card. The chart board gives each card one question and the chart form that answers it best: a split bar for income by source, a dot matrix for goals saved, a spline for this week's daily spend, mirrored bars for money in above the line and out below across twelve months, stacked blocks for spent versus saved, pace bars that mark where a budget *should* be today, and a calendar heatmap on a square-root scale so one large day does not wash out the month. There is no chart library: eighteen chart components are hand-built as SVG or plain HTML, sharing one tokens file (line width, bar thickness, gaps, entrance timing), one Catmull-Rom smoothing function and one entrance hook. The figures come from a pure module, `domain/dashboardCharts`, tested without a browser.

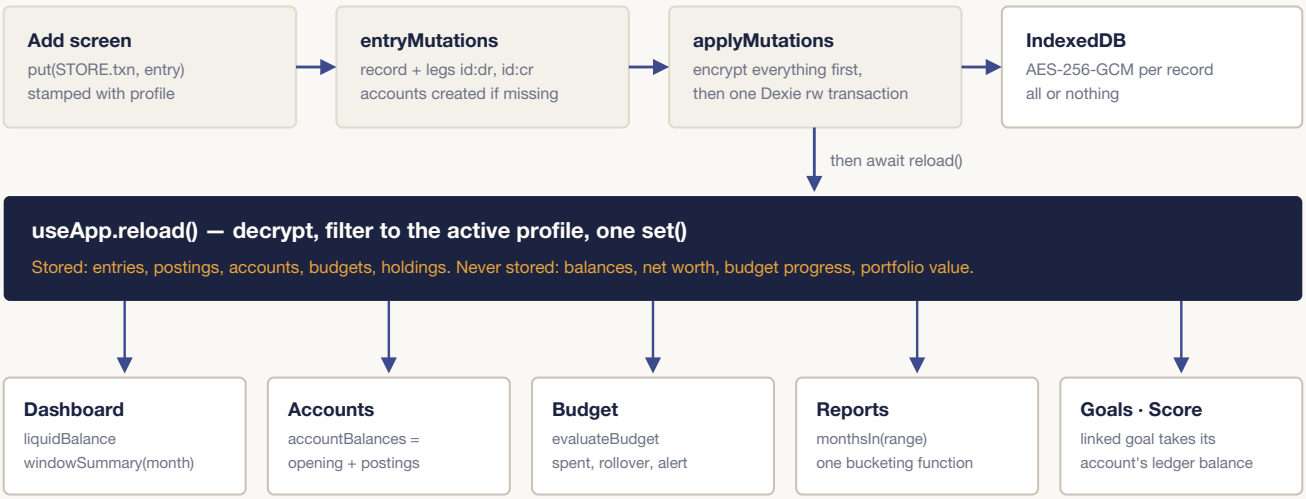
Arranged by the user. Every card is one of 28 widgets on a 12-column grid that drops to 6 columns on a tablet and 1 on a phone. Customize lets the user drag a card by its handle, choose S, M, L or XL, or hide it. Three details make it hold up:

- `resolveLayout()` treats a saved layout as untrusted input: it drops unknown or duplicate ids, clamps sizes to each widget's minimum, and slots widgets added in later releases next to their default neighbours — so an update never strands a new card or crashes on an old one.
- `packSpans()` fills rows in order, and when a card does not fit, the last card of the row it leaves widens to close the gap. Whatever the user picks, no row ends in empty space.
- Each card is a CSS *container*, so its contents lay out by the card's own width rather than the window's — Net worth shows its parts two, three or five across depending on how big the user made it.

The drag handle also takes arrow keys, the size buttons have spoken names, and card contents are `inert` while editing so a drag cannot click a link. The layout is saved in local storage per profile, not in the vault: it is a preference about this screen on this device, and putting it in the encrypted ledger would make it travel in backups to devices with different screens.

One source of truth — how a write reaches every screen

The web client keeps exactly one in-memory copy of the vault: a Zustand store. There are no per-screen caches, no live queries and no event bus. Every write follows the same path, and every screen re-derives from the same arrays afterwards.



Every screen subscribes with a selector and recomputes in useMemo. One write, one reload, one set — so no screen can be a write behind another.

The write path on the web client. The detail that matters most is the order inside `applyMutations`: encryption happens before the database transaction opens, because awaiting a non-Dexie promise inside a Dexie transaction lets it commit early — the obvious version would look atomic and not be.

Six rules keep the screens in agreement:

1. **Derive, do not store.** A balance is an account's opening balance plus its postings, computed on read. Net worth, cash, budget progress and portfolio value are the same. A stored total is a second copy of the truth that can go stale; the only stored aggregate is the daily net-worth snapshot, which is history by definition.
2. **One function per figure.** "Cash" is `liquidBalance(accounts, postings)` wherever it appears — the balance hero, Accounts, Forecast, Score and the snapshot. When the dashboard's net-worth card once summed transactions instead, it disagreed with the card beside it — opening balances were missing from one and not the other. A test now pins the two together, with an opening balance in the fixture because that is exactly what the two methods disagree about.
3. **Money is a decimal string end to end.** Stored as text, computed with `decimal.js` at 30 digits, parsed tolerantly so one malformed amount renders as zero with a warning rather than crashing a screen. The ledger is in rupees; the display currency is applied only at the edge by one hook, `useFmt`, with Indian digit grouping for INR.
4. **One date format and one period model** — the next subsection.
5. **Profile scoping happens once.** `reload()` filters every record type to the active profile, and new writes are stamped with it. No screen filters by profile itself, so no screen can forget to.
6. **Privacy masking is a property of the money primitive.** Ghost mode passes `hidden` to `MoneyValue`, which renders bullets, and chart labels and screen-reader text say "hidden". It is not a CSS blur, because a blur leaves the real figure in the DOM, in the accessibility tree and in a screenshot.

The Flutter client reaches the same guarantee by a different route: Drift's `.watch()` queries exposed as Riverpod `StreamProvider`s, so the database itself pushes changes to every listening widget. Same contract, different mechanism — each one idiomatic for its platform.

Time periods

Every period in the web client is one immutable type from `domain/period.ts`: a `DateRange` with an inclusive start and end in local time, a kind (month, trailing, financial year or custom) and a **label that travels with it**. The caption a screen prints is the label of the range it queried, so the words and the numbers cannot disagree. That rule came from a

defect: the dashboard once computed "this month" as the trailing thirty days, which on 9 August silently counted three weeks of July into a figure labelled August.

- Months are calendar months, never thirty-day spans; `monthsBack(n)` steps by calendar month.
- The financial year is India's, starting 1 April. Tax rules are chosen by the disposal date, not today's date.
- `monthsIn(range)` is the one bucketing function every monthly chart uses, so two charts of the same period always have the same bars.
- Dates are built from local parts, never via `toISOString`, which would shift a date by a day for anyone east of UTC.
- `useNow` ticks once a minute, so a tab left open across midnight or a month end rolls into the new period on its own.
- Recurring rules clamp to month end: a bill on the 31st falls on the 28th in February rather than skipping into March.

The period lives in each screen's own state rather than in a global selector, deliberately. A budget is read a month at a time, reports compare ranges, and the net-worth chart wants years. One global period would force each screen to reinterpret a choice made for a different screen. What is shared is the *model*, not the selection. The honest gap: the Flutter dashboard's 1M and 3M windows are still 30- and 90-day spans rather than calendar months.

Keeping three artefacts in step

The tokens exist in three places: the web stylesheet, the Flutter theme, and an 87-sheet design specification drawn from the tokens rather than from screenshots. `sync-tokens.mjs` copies the token blocks out of `globals.css` into the specification's stylesheet, and `check-tokens.mjs` fails if a colour in the app is missing from the spec or the categorical palette order differs. Web to Flutter has no code generation; it is held by mirrored contrast tests and by the twin component kits (`ui.tsx` and `khazana_cards.dart`, `charts/tokens.ts` and `chart_tokens.dart`). That is the weakest link, and Section 17 lists where it has drifted.

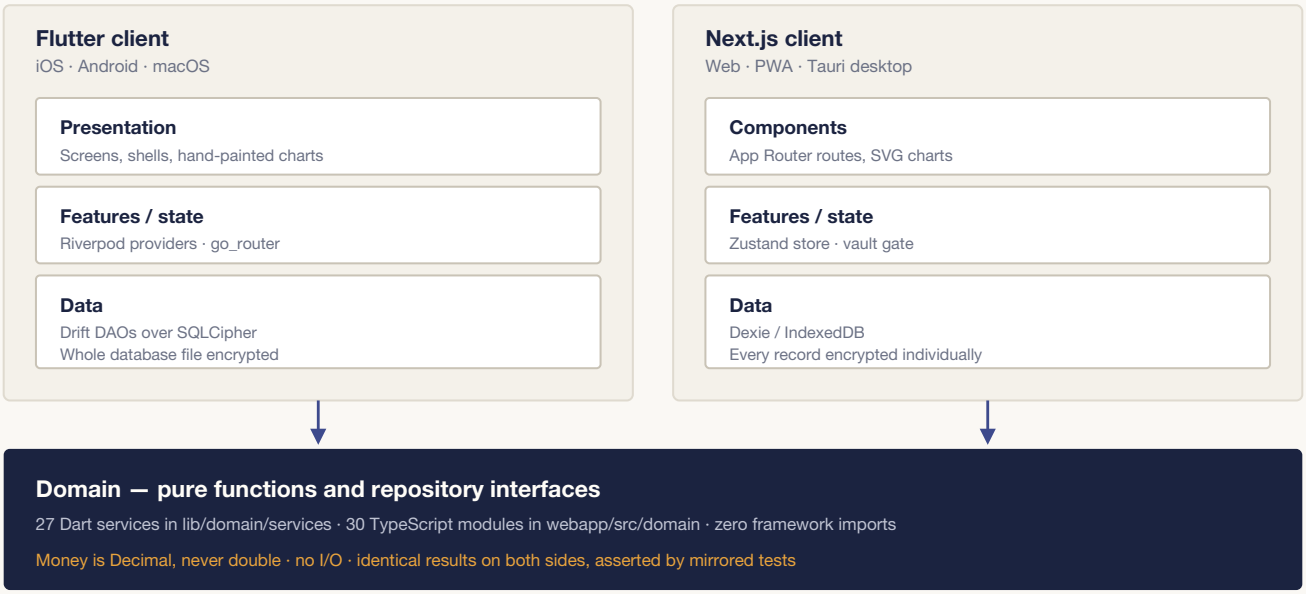
Shipped since the previous edition

CHANGE	WHAT IT DOES, AND THE DECISION INSIDE IT
Chart board	The board described above, a different chart form on each card, with its figures in a tested pure module.
Arrange the dashboard	28 widgets, drag and resize, per-profile layout, gap-free rows, container-sized contents.
Profile photo	Cropped to a square with a 20% upward bias so tall portraits keep the face, scaled to 256px and re-encoded as JPEG — which drops EXIF, including location — and stored inside the encrypted profile record.
Update banner	Each build stamps an id; open tabs poll for it. An unlocked tab shows a banner and never reloads by itself, because reloading drops the key and asks for the PIN. A locked tab has nothing to lose and reloads straight away.
Edit a recurring rule	Saves under the same id and leaves past transactions untouched; the next run date is kept exactly when the date field is unchanged.
Firefox line chart	The draw-in used a `pathLength` stroke dash, which Firefox measures differently on a stretched viewBox, ending the line two-thirds across. Now a clip reveal that every engine draws the same.
Visual regression	Playwright screenshots of ten routes in four projects — desktop and phone, light and dark — with the clock frozen, UUIDs replaced by a counter, the network blocked and motion reduced. Baselines are generated on Linux by CI.

Section 06

System architecture

One product, two independently implemented clients, four rings each, and a domain core that imports no framework at all.



Dependencies point inward. Nothing in the domain layer knows that Flutter, React or SQLite exist.

The layered model, held identically on both clients. The value of the inward-pointing rule is concrete rather than aesthetic: because the domain layer has no I/O and no framework imports, it was portable from Dart to TypeScript by hand, and it is testable without a database, a widget tree or a browser.

Why the domain layer is framework-free

The rule sounds academic until you try to ship the same product twice. A health score that reaches into a Riverpod provider cannot be ported; a net-worth calculation that queries a DAO cannot be tested without a database. By keeping the domain as plain functions over plain entities, three things became possible at once: the logic ports across languages, it tests in microseconds without fixtures or mocks, and — the point that matters commercially — both clients can be proven to compute the same number for the same input.

The stack, concretely

CONCERN	FLUTTER CLIENT	NEXT.JS CLIENT
Language	Dart 3.10	TypeScript 5
UI	Flutter 3.38, Material with a custom design system	React 19, Tailwind v4 with CSS-native theming
State	Riverpod 2, manual provider wiring for DI	Zustand 5, one store as the whole session
Routing	go_router 17 with a vault-state redirect guard	Next.js App Router, static export
Persistence	Drift 2.2x over SQLCipher, generated by build_runner	Dexie 4 over IndexedDB
Crypto	pointycastle for PBKDF2, package:cryptography for AES-GCM and SHA-256	Web Crypto for PBKDF2 and AES-GCM

CONCERN	FLUTTER CLIENT	NEXT.JS CLIENT
Money	package:decimal	decimal.js at 30-digit precision
Charts	Six hand-written CustomPainters	Nine hand-written SVG components
Packaging	App Store, Play Store, Mac App Store, signed DMG	Static export, PWA, Tauri 2 for Windows, macOS and Linux

DELIBERATE NON-DEPENDENCIES

There is no charting library on either side, because every chart needed a pixel-comparable twin in the other language and a library would have made that impossible. There is no web font fetched at runtime — Inter is bundled, so the app never touches the network to render. And there is no analytics, crash-reporting or telemetry SDK anywhere in either dependency tree; that absence is verified by inspection, not asserted in a privacy policy.

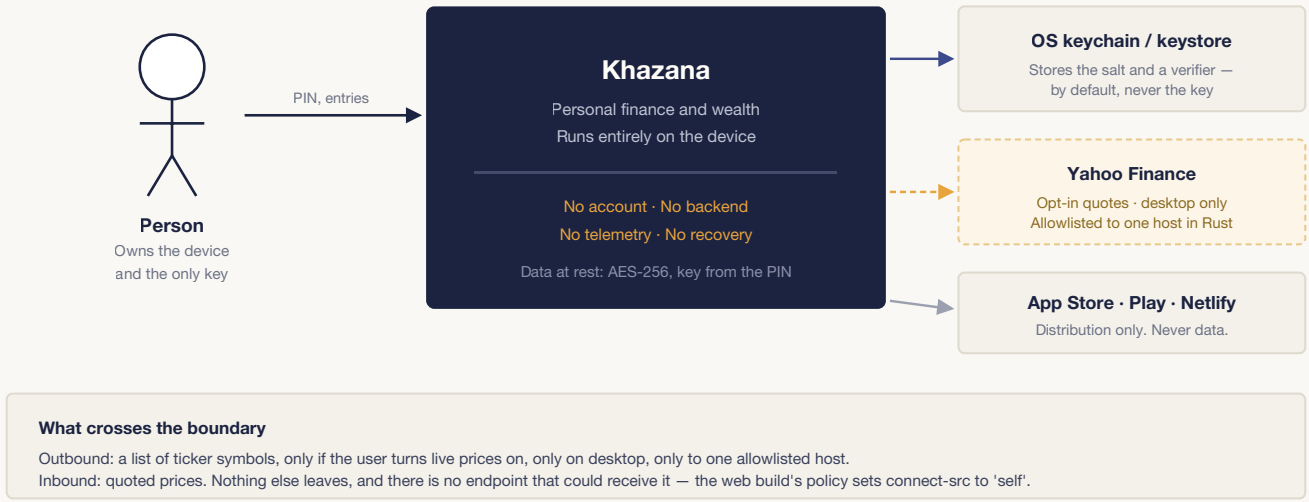
Section 07

Architecture at four levels

The same system drawn at four magnifications — context, container, component and code. Each level answers a question the level above it raises and the level below it cannot see.

L1 System context

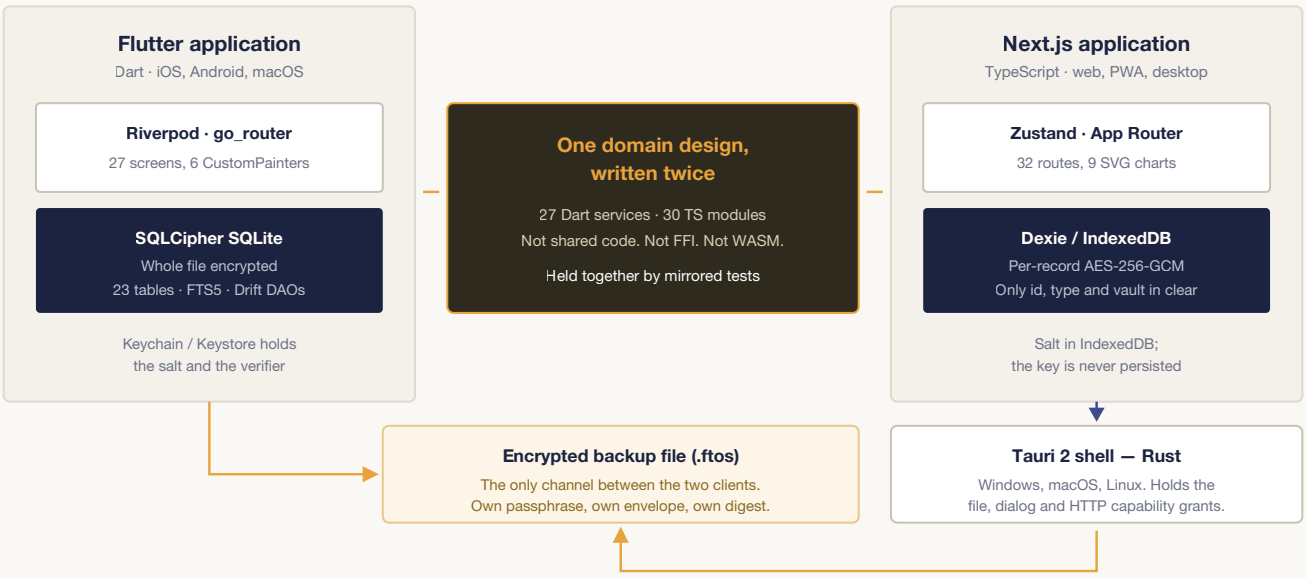
The context diagram of a normal finance application is busy: an API gateway, an identity provider, a database cluster, an aggregator such as Plaid, an analytics pipeline, a payment processor. Khazana's is almost empty, and that emptiness is the product.



L1 — system context. Three external actors, of which one is optional, one is the operating system and one is a distribution channel. The single dashed edge is the entire network surface of the product.

L2 Containers

Five deployable things, and — the point of the diagram — *no arrow between the two clients*. They share source-level design and a file format, not a connection. Two devices reconcile by one exporting an encrypted backup and the other importing it, which is why the merge had to be a deterministic function rather than a protocol.



L2 — containers. Five deployables, one file format, zero network links between them.

L3 Components

Inside each client, four rings with dependencies pointing strictly inward — the model already drawn in Section 06. What that section does not show is the *enforcement*: the domain ring imports no framework, performs no input and output, and takes no clock. Time is passed in, which is why a forecast can be tested at an arbitrary date without mocking a system call.

RING	FLUTTER	NEXT.JS
Presentation	`lib/presentation`, `lib/features/*/screens`	`src/app`, `src/components`
State	`lib/features/*/providers` — Riverpod	`src/lib/store.ts` — one Zustand store
Data	`lib/data` — 16 DAOs, 13 repositories	`src/lib/repo.ts`, `db.ts`
Domain	`lib/domain` — 27 services, repository interfaces	`src/domain` — 30 modules

The dependency inversion is real rather than decorative: the domain ring declares repository *interfaces*, and the data ring implements them. A health score depends on the shape of an account, never on Drift.

L4 Code

At the innermost level the diagram stops being useful and the type signatures take over. Four of them carry most of the system's guarantees, and they are set out in Section 08.

Section 08

High-level and low-level design

The design as it would be handed to another engineer: first the decomposition and the reasoning, then the contracts precise enough to implement against.

High-level design

The whole system falls out of one constraint that was fixed before any code was written: **no server, ever**. Not a light one, not one "just for sync", not one "just to validate licences". Everything below is downstream of that refusal.

PROBLEM A SERVER USUALLY SOLVES	HOW IT IS SOLVED INSTEAD	WHAT THAT COST
Authentication	A PIN stretched into the encryption key. There is no account, so there is no session, token, refresh or revocation.	No recovery. A forgotten PIN is a destroyed vault, stated plainly rather than hedged.
Multi-device sync	Deterministic last-write-wins merge over exported backups, with 90-day tombstones so a delete is not undone by a stale copy.	Not real time. The user carries the file, and the merge must be symmetric — same result whichever side runs it.
Schema migration	Idempotent migrations with deterministic identifiers, running on a database that can never be inspected or rolled back centrally.	Every migration must be correct the first time, on data nobody has ever seen.
Crash reporting	An encrypted local ring buffer, drained into a log database once the vault opens. User-initiated, never transmitted.	No aggregate view of failures across users. The trade is deliberate: an error message here can quote a merchant and an amount.
Price data	One allowlisted host, opt-in, desktop only, routed through Rust because the endpoint sends no permissive headers.	Mobile and browser users enter prices by hand or import them.

Module decomposition follows the same logic. The domain ring is split by financial concern rather than by screen — `health`, `forecast`, `tax`, `lots`, `reconcile`, `statements`, `safetyNet`, `recurrence`, `analytics` — because a screen is a presentation decision and a concern is not. Several screens read `health`; none of them own it.

Low-level design

Four contracts carry a disproportionate share of the system's correctness.

1 · Money is text, and aggregation happens in the language

```
// lib/data/database/converters.dart
class DecimalConverter extends TypeConverter<Decimal, String> {
  Decimal fromSql(String fromDb) => Decimal.parse(fromDb);
  String   toSql(Decimal value)  => value.toString();
}
```

Money columns are `TEXT`, never `REAL`. The consequence is followed through rather than forgotten: because the column is text, `SUM()` would sort and add lexicographically, so no query in the codebase uses it. Every DAO returns rows, and totals are computed in Dart over `Decimal` — or in TypeScript over `decimal.js` at thirty digits of precision. The web-side parser is deliberately tolerant, stripping currency symbols, group separators, non-breaking spaces and accounting

parentheses, and returning zero with a warning rather than throwing: it is called during render, so one malformed record used to take a whole screen down on every paint. The record that needed rescuing is then reported by the diagnostics check rather than quietly forgiven.

2 · A journal entry and its legs commit together or not at all

```
// webapp/src/lib/repo.ts
export async function applyMutations(key, vaultId, mutations: Mutation[]) {
  // Encryption happens up front, deliberately outside the transaction:
  // awaiting a non-Dexie promise inside a Dexie transaction drops out of
  // its zone and the transaction commits early, defeating the point.
  await db.transaction('rw', db.records, async () => {
    if (deletes.length) await db.records.bulkDelete(deletes);
    if (puts.length)    await db.records.bulkPut(puts);
  });
}
```

A transaction and its two ledger legs are one financial event. They were once three sequential writes, and a failure between them left an entry with a single leg — an unbalanced ledger, which is the one state the whole double-entry model exists to make impossible. The comment about zones is the load-bearing part: the naive fix, awaiting the encryption inside the transaction, would have looked correct and silently committed early.

3 · Migrations are idempotent because they can never be inspected

```
// lib/data/database/app_database.dart — schemaVersion = 6
onUpgrade: (m, from, to) async {
  if (from < 2) { await m.createTable(insurances); await m.createTable(netWorthSnapshots); }
  if (from < 3) { await _migrateToDoubleEntry(m); }        // single entry → two legs
  if (from < 4) { await _migrateToLotLevelPortfolio(m); }  // aggregate rows → lots
  if (from < 5) { await m.addColumn(netWorthSnapshots, netWorthSnapshots.healthScore); }
  if (from < 6) { await m.createTable(notificationDeliveries); }
}
```

Version 3 is the interesting one: it converts every single-entry transaction into a balanced pair and seeds a chart of accounts using *deterministic* identifiers — `acct-cash-$vault`, `acct-exp-$category` — so re-running it cannot create a second cash account. New columns are nullable by default, and the health-score columns specifically refuse to back-date invented history: a vault that has no score for last March shows nothing there, not zero.

4 · The backup envelope is a format, not a serialisation

OFFSET	FIELD
0x0000	Magic `FTOS` — so a wrong file is rejected as a wrong file
0x0004	Envelope version (currently 3; versions 1 and 2 still restore)
…	Schema version, app version, 64-bit timestamp, plaintext size
…	32-byte SHA-256 of the plaintext, compared in constant time
…	12-byte GCM nonce, then ciphertext, with the 16-byte tag at end of file

A wrong passphrase and a tampered file return the *same* error, deliberately. Distinguishing them would turn the restore screen into an oracle for guessing passphrases against a stolen backup. Restore validates the file's contents before writing anything, applies the whole thing in one transaction, and remaps incoming profile identifiers onto the receiving device's own — the omission that once wrote 367 intact records where no screen could see them.

Algorithms worth naming

ALGORITHM	CHOICE AND THE REASON FOR IT
Key derivation	PBKDF2-HMAC-SHA256, 600,000 iterations, 32-byte salt. Off the UI isolate on native; native Web Crypto on the web.
FIFO lot matching	Not configurable. Choosing an accounting method you are not entitled to produces a return you cannot file.
XIRR	Bisection over –99.99% to 1000%, 200 iterations, tolerance 1e-7. Newton–Raphson is faster but diverges on the sign-changing series real portfolios produce.
Recurrence	`advance()` clamps month ends. Naive date arithmetic overflows — 31 January plus one month becomes 31 February — so a rule dated the 31st skipped February and posted twice in March.
Merge	Last-write-wins per record with 90-day tombstones, symmetric by construction so both devices reach the same state.

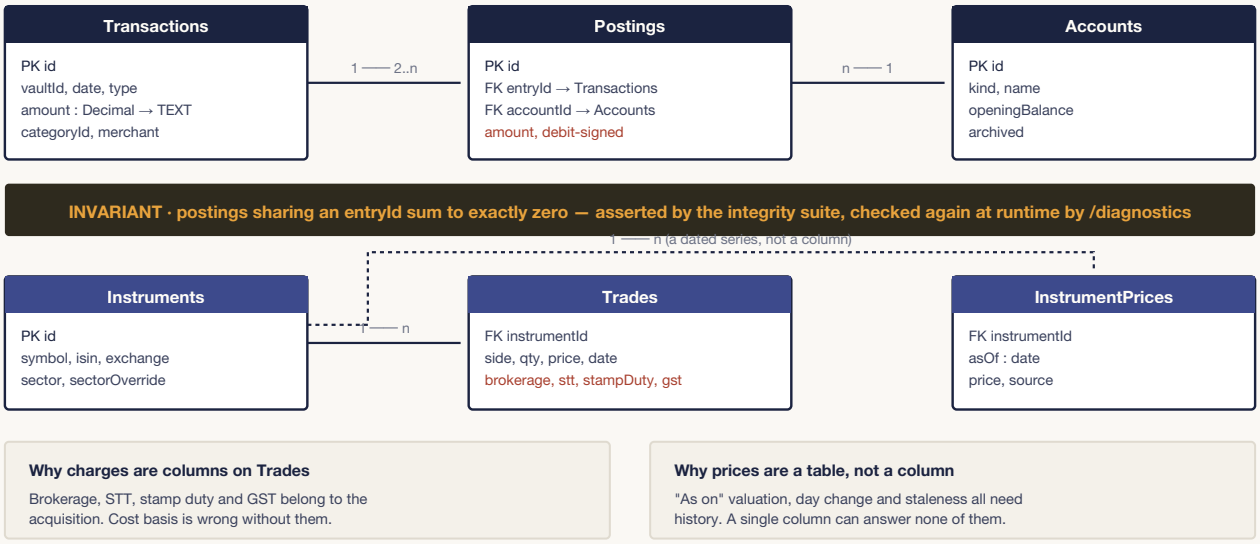
Section 09

The model in UML

Four diagrams, each included because it states something the prose around it cannot: a shape, an ordering, or a set of states and the transitions between them.

Entity relationships — the ledger core

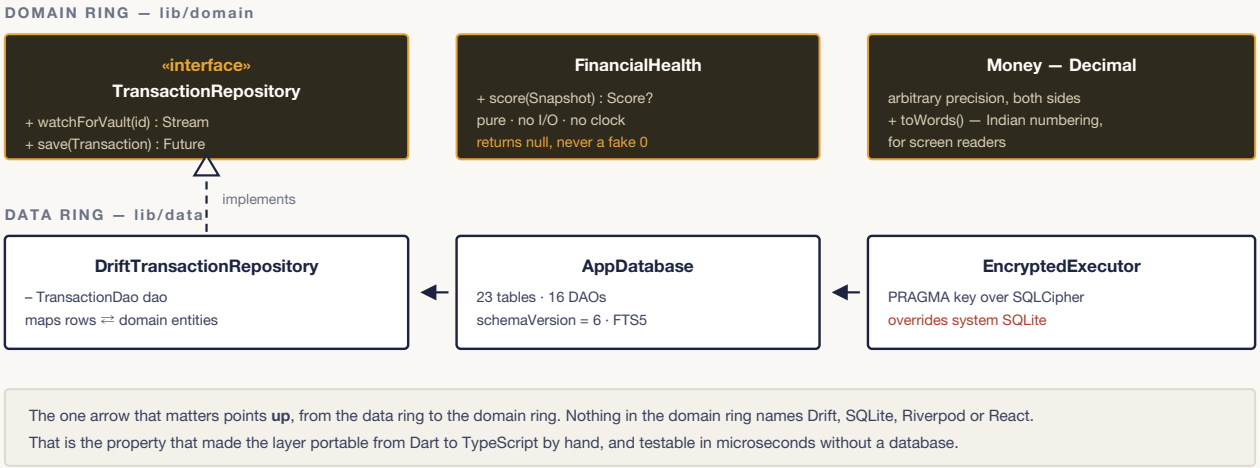
Twenty-three tables is too many to draw legibly and mostly uninteresting. What matters is the spine: a journal entry owns postings that must sum to zero, and a lot-level portfolio hangs off instruments rather than off aggregated positions.



The remaining seventeen tables — budgets, goals, liabilities, insurance, snapshots, recurring rules, dividends, fund holdings, benchmarks, captures, fingerprints, aliases, FX rates, notification deliveries — hang off this spine without complicating it.

Entity relationships. The amber band is the invariant the double-entry design exists to protect; everything above it is structure, and everything below it is the portfolio model that replaced aggregated positions in schema version 4.

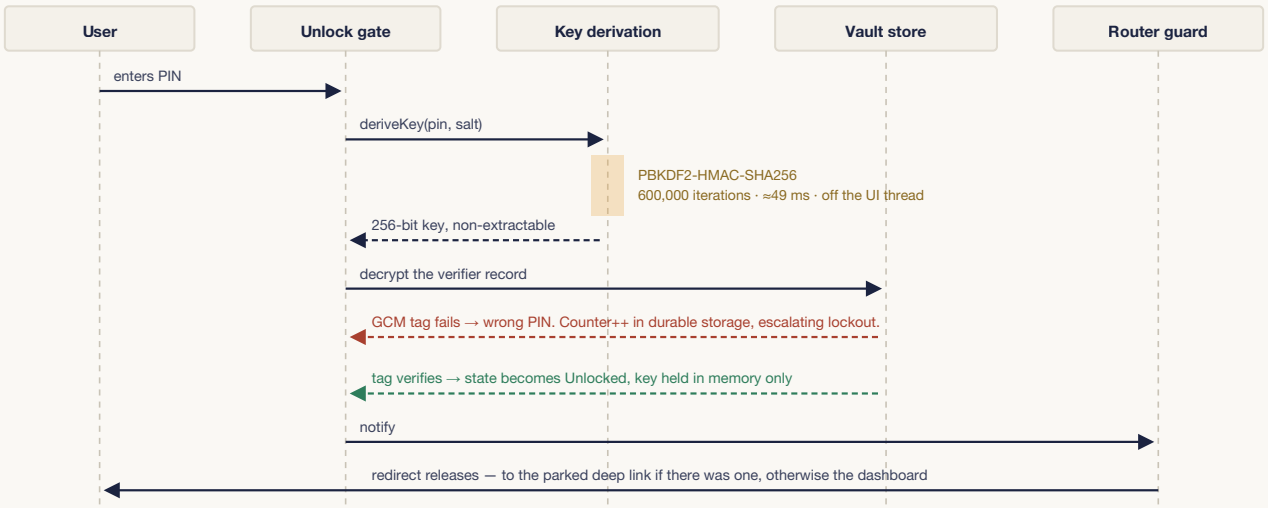
Class structure — dependency inversion across the rings



The one arrow that matters points **up**, from the data ring to the domain ring. Nothing in the domain ring names Drift, SQLite, Riverpod or React. That is the property that made the layer portable from Dart to TypeScript by hand, and testable in microseconds without a database.

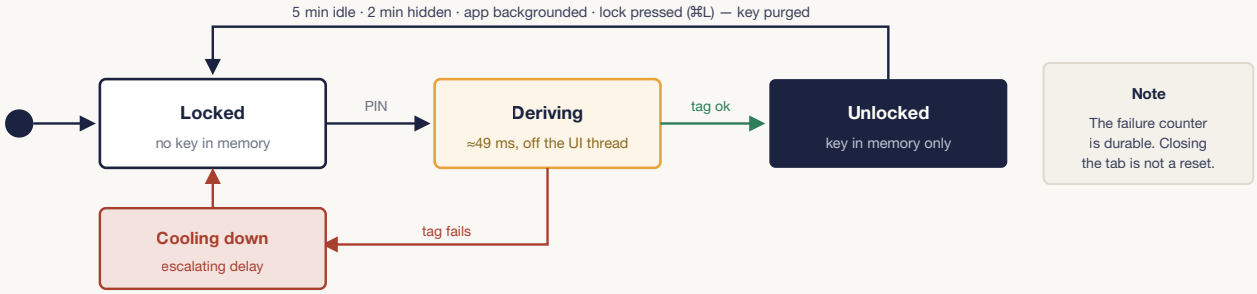
Class structure, abbreviated to the relationship that carries the design. The dashed hollow-triangle edge is a realisation: the implementation depends on the interface, never the reverse.

Sequence — unlocking a vault



Unlock. The PIN is never stored and never compared; it is used to derive a key, and the key's correctness is established by whether an authenticated-encryption tag verifies. A failure is indistinguishable from a corrupted record by design.

State — the vault lifecycle



There is no "logged in" state that survives a restart, because there is no session — only a key that exists in memory or does not.

Vault states. Every path out of Unlocked purges the key, and the only path back in re-derives it from the PIN.

Section 10

Data and persistence

23	v6	FTS5	TEXT
DRIFT TABLES	SCHEMA VERSION	SEARCH INDEX	MONEY STORAGE TYPE

Money is never a floating-point number

This is the single non-negotiable rule of the codebase, and it propagates further than people expect. Money is an arbitrary-precision decimal end to end — `package:decimal` in Dart, `decimal.js` in TypeScript — and it is stored in SQLite as `TEXT` through a custom type converter, never as `REAL`. The consequence that has to be handled deliberately: because the column is text, numeric aggregation and sorting happen in Dart rather than lexicographically in SQL.

Parsing is deliberately tolerant rather than strict. Real inputs arrive with non-breaking spaces, zero-width characters, currency symbols, thousands separators and accounting parentheses for negatives. The parser recovers from all of them, warns, and returns zero rather than throwing — because a single malformed row used to take down an entire screen. Rounding rules are explicit and local: tax stays in decimal until the final step, EMI is admitted to be irrational and is computed in floating point then rounded once, and decimal division uses a fixed scale rather than an implicit one.

Two storage models, one guarantee

	FLUTTER — DRIFT OVER SQLCIPHER	WEB — DEXIE OVER INDEXEDDB
Unit of encryption	The entire SQLite database file	Each record individually
Mechanism	`PRAGMA key` issued as the first statement, using the bundled SQLCipher library	AES-256-GCM with a fresh 12-byte IV per record
Plaintext on disk	None	Only `id`, `type` and `vaultId` — the index keys
Wrong key behaviour	A probe query fails fast and raises a typed "wrong credentials, file untouched" error rather than "your file is corrupt"	The GCM authentication tag fails and the record is skipped

A detail worth naming in an interview: the SQLCipher library override is pinned to the bundled binary on every platform specifically because iOS and macOS ship their own SQLite, and letting the system library shadow the bundled one produces a database that silently writes plaintext. That failure mode is invisible — everything works, and nothing is encrypted.

The schema, and how it got to version six

Twenty-three tables plus an FTS5 virtual table with a porter tokenizer and three synchronisation triggers, in a single encrypted database; error logs live in a second, separately encrypted database so that a crash-logging bug can never corrupt the ledger. Every table carries a `vaultId` for multi-vault isolation and a `profileId` where the entity is profile-scoped.

VERSION	MIGRATION
v2	Insurance policies and daily net-worth snapshots.

VERSION	MIGRATION
v3	**Double-entry accounting.** Adds accounts, postings and pending captures, then converts every existing single-entry transaction into a balanced two-leg journal entry using deterministic account identifiers. The migration is idempotent, because a half-applied accounting migration is worse than none.
v4	**Lot-level portfolio.** Adds instruments, trades, prices, dividends, fund holdings and benchmark series. Before this, holdings were one aggregated row per symbol with a single average cost and no sector — profit and loss was not computable at all.
v5	Health score and tracked weight recorded on each snapshot — nullable, with no back-dating of scores that were never actually computed.
v6	A notification delivery log, so reminder cooldowns survive a restart.

Search

```
CREATE VIRTUAL TABLE transactions_fts USING fts5(
  txn_id UNINDEXED, content, tokenize='porter unicode61');
```

Triggers keep the index in step with inserts, updates and deletes, so search can never go stale. Query strings are escaped before reaching SQLite — that escaping is named in the threat model as the injection control, since it is the one place user text becomes query syntax.

Multi-device reconciliation without a coordinator

Two devices reconcile their encrypted backups with no server involved. The merge is last-write-wins keyed on each record's `updatedAt`, with tombstones retained for ninety days so a deletion propagates instead of being resurrected by the other device's stale copy. The function is symmetric — it runs identically on both peers and produces the same result regardless of which side runs it — and on an exact timestamp tie, delete wins.

Section 11

Security and privacy engineering

The security story here is applied client-side cryptography, not infrastructure hardening. There is no server, no account and no network trust to reason about. The whole model reduces to one sentence: on-device data must be unreadable without the user's PIN.

The key lifecycle

```
PIN + 32-byte salt  ──PBKDF2-HMAC-SHA256, 600,000 iterations──▶  256-bit AES key
```

1. **Derive.** On native the derivation runs off the UI isolate so unlock does not jank; on web it uses Web Crypto. Both are standard PBKDF2 and produce the same key from the same inputs.
2. **Use.** Mobile opens SQLCipher with the key as the very first statement, followed by a probe query that fast-fails on a wrong key. Web derives a *non-extractable* AES-GCM key object and encrypts each record with a fresh IV.
3. **Store.** Only the salt is persisted. The key itself is never written anywhere in the default flow.
4. **Purge.** On lock — five minutes idle, tab hidden, or app backgrounded — the in-memory key and all decrypted data are cleared, and routing falls back to the gate.

Verifying a PIN that was never stored

At setup the app encrypts a known token with the derived key and keeps the ciphertext. On unlock it derives a key from the entered PIN and attempts to decrypt that verifier. Because AES-GCM is authenticated, a wrong key fails the authentication tag and throws — so a wrong PIN simply cannot produce a plausible decryption, and no data is exposed in the attempt. On mobile the equivalent is the SQLCipher probe query.

What the crypto defends against

THREAT	DEFENCE
Attacker copies the database at rest	AES-256 at rest on both platforms; without the key the file is ciphertext.
Brute-forcing the PIN into the key	PBKDF2-HMAC-SHA256 at 600,000 iterations with a 32-byte salt, plus an exponential unlock lockout checked *before* the derivation runs, so an attacker cannot use the rate limiter to consume our CPU.
Tampering with ciphertext or a backup file	AES-GCM authenticated encryption; the tag is verified on every read.
Forged or corrupt backup restore	A six-gate validation: magic header, envelope version, structure, decryption, SHA-256 digest compared in constant time, then schema version.
Turning the restore screen into an oracle	A wrong PIN and a tampered file report the same error deliberately, so an attacker cannot learn which of the two they hit.
Key lingering in memory	Purged on inactivity, tab-hide and app-background.
Sensitive values leaking into logs	A sanitiser redacts any run of four or more digits before a log line is written, leaving short values like status codes intact.
Shoulder-surfing	Ghost mode masks every monetary value with one toggle.

THREAT	DEFENCE
Exfiltration from the web client	A content-security policy whose load-bearing line is `connect-src 'self'` — this client fetches nothing, so anything attempting to send a decrypted vault to another origin is blocked outright.

The decision I am most proud of: removing a key from the keychain

The original design stored the derived key in the OS keychain so biometrics could unlock without a PIN. That broke on macOS in a way that took real debugging: keychain items are bound to the code signature that created them, so every rebuild turned the app into a stranger asking for another application's secret, surfacing as `errSecMissingEntitlement (-34018)`.

The fix could have been a keychain configuration tweak. Instead I changed the model: the default path now stores only the salt and a domain-separated SHA-256 verifier of the derived key, and recomputes the key from the PIN at every unlock. Biometric unlock became opt-in, storing the key only when the user explicitly asks for it. **The result is that in the default flow there is no key at rest at all** — a security improvement that arrived through a usability failure, which is the honest way most of them arrive.

Platform hardening

- **Android** — `allowBackup="false"` and explicit data-extraction rules, because the platform default silently uploads app data. `READ_SMS` and `RECEIVE_SMS` are absent by choice; the notification listener covers the same alerts without a restricted permission.
- **macOS** — sandboxed, with only outbound network client access and user-selected file access. The network *server* entitlement is deliberately absent: the app never listens.
- **iOS and macOS** — Face ID usage strings and a declared non-exempt-encryption flag, both verified in CI so a store rejection cannot come as a surprise.
- **Keychain** — one pinned configuration used everywhere, set to first-unlock-this-device and explicitly non-synchronisable, so secrets survive a reboot but never reach iCloud or another device. Deletion overwrites before removing.

What it deliberately does not defend against

Stating this clearly is part of the security posture, not a caveat to it. A compromised device — jailbreak, keylogger, or malware running as the user — can observe the PIN or decrypted memory while the vault is open. A forgotten PIN means the data is gone; there is no recovery, no escrow and no reset, which is the price of the guarantee. Coercion is out of scope. Server-side concerns are not applicable because there is no server. And the side-channel resistance of SQLCipher, Web Crypto and pointycastle is relied upon as provided rather than independently verified.

Zero telemetry, and what replaces it

There is no analytics, crash-reporting or product-metrics SDK in either client. The reasoning is specific: a stack trace from a finance app is a description of what the user was doing when it broke, and I have no business holding that. The consequence is that the only crash report I will ever see is one a user chooses to send from Settings.

What stands in its place is a local crash guard that installs handlers for framework errors, uncaught platform errors and async gaps, and buffers up to fifty scrubbed records in memory — because a crash loop can throw thousands of times a second, and the first few explain the last few. Once the vault opens, the buffer drains into an encrypted log database with a thirty-day retention. Records are scrubbed *before* entering the buffer, so nothing sensitive sits in memory either. The replacement error screen tells the user the one thing they actually need to know: their data is safe and unchanged.

Section 12

The financial engine

Twenty-seven pure Dart services and thirty TypeScript modules, none of which perform I/O. This is the part of the project I would most want to be asked about.

Lot-level portfolio and FIFO matching

Sells consume the oldest open lots first, ordered by trade date with the identifier as a deterministic tie-break so the same input always produces the same match. Proceeds are computed net of the sale's charges, pro-rated so a partial match carries its share. Two sign conventions matter and are easy to get backwards: **charges add on a buy and subtract on a sell**. Reversing that is the standard way profit and loss ends up reading better than reality.

Selling more than was ever bought is skipped rather than turned into a negative or short lot. FIFO is deliberately not configurable, on the grounds that choosing a method you are not entitled to produces a return you cannot file.

> **THE INVARIANT THAT ANCHORS THE WHOLE MODULE**
>
> A roll-up along *any* dimension — sector, industry, market cap, asset group, asset type, instrument, currency — must sum exactly to the portfolio total. Unclassified values become an explicit bucket and are never dropped. This is asserted in both the Dart and the TypeScript test suites, and the rule when it fails is stated in the source: if the reconciliation test breaks, the roll-up is wrong, not the test.

Honest nulls — a theme rather than a detail

A recurring design decision across the engine is to return *nothing* rather than a plausible-looking zero:

- Profit-and-loss percentage is `null`, rendering as an em dash, when cost is zero — never 0%.
- XIRR returns `null` when there is no sign change or fewer than two cash flows, rather than a fabricated rate.
- Concentration is `null` on an empty book, not 0.
- Safe-to-spend returns a typed gap — "no income recorded", "no history" — rather than inventing an allowance.
- The financial-health grade is withheld entirely below fifty points of tracked weight, because a letter grade computed from a quarter of the picture is a lie with a serif on it.
- Unpriced holdings are counted at cost, contribute zero profit and loss, and every roll-up row carries an `unpricedCount` so the interface can disclose that its own number is understated.

XIRR by bisection

Money-weighted return is found by bisecting the net-present-value function over a rate range of −99.99% to 1000%, with 200 iterations and a tolerance of 1e-7. Portfolio flows are buys negative, sells and dividends positive, and current market value as a terminal flow. Bisection rather than Newton–Raphson is a deliberate robustness choice: Newton converges faster but can diverge on the irregular, sign-changing cash-flow series a real portfolio produces, and a wrong return is worse than a slow one.

Double-entry accounting

Postings are debit-signed and every journal entry sums to zero. Net worth is computed from asset and liability balances plus the market value of holdings, structured so nothing is counted twice — transfers debit the destination and credit the source, so they cancel. The balance sheet derives two lines the ledger does not store: retained earnings, as cumulative income minus expense, since income and expense accounts are never closed; and opening capital, as the contra for

account opening balances. With both present, assets plus liabilities plus equity is identically zero — which means any non-zero result is real damage, and that is exactly what the diagnostics screen checks.

Capital-gains tax

Rules are a versioned data set selected *by disposal date*, so a sale made before July 2024 is not taxed under rules that did not exist when it happened. Where the short-term rate is slab-based the engine reports it as slab rather than guessing a number, and surfaces slab-rate gains separately so the headline figure cannot understate the bill. The most interesting piece of modelling is the annual exemption:

```
// The ₹1.25 lakh long-term exemption is ONE allowance per financial year,
// not per holding. It is pooled and spent largest-gain-first, which is both
// optimal for the user and deterministic — the same book always produces
// the same allocation.
```

Per-asset treatment is modelled rather than approximated: foreign equity is not eligible for the concessional long-term regime because no securities transaction tax was paid; a sovereign gold bond redemption is a maturity event and not a disposal; fixed-income instruments with no quoted price are valued as principal plus accrued interest, compounding quarterly for cumulative deposits per Indian convention, with accrual stopping at maturity.

Weighted scoring, and the choice not to punish absence

The financial-health score runs over four categories — Wealth 30, Protection 25, Efficiency 25, Future 20 — each composed of weighted metrics. The design decision that makes it usable:

```
// A metric's value is null — never 0 — when the user has not tracked it.
// Untracked metrics leave the DENOMINATOR rather than scoring zero.
const score = Math.round((tracked.reduce((s, c) => s + c.score, 0) / trackedWeight) * 100);
const grade = trackedWeight >= MIN_GRADABLE_WEIGHT ? gradeOf(score) : null;
```

A user who has not entered insurance is not failing at insurance — they are not tracking it, and a score that conflates the two teaches people to distrust the number and stop looking. Related touches: a flat net worth scores 0.33 on trajectory rather than zero, because holding steady is not failure; the debt component is multiplied by 0.85 when any interest rate exceeds 24%; goal pace excludes emergency-fund goals because those are Protection's business; and Protection reuses the Safety Net components directly rather than recomputing them, so the two scores cannot disagree.

Colour as a computed property

The allocation charts have to separate fourteen asset types, which is far more categories than can be reliably distinguished by hue. Rather than assert that a palette is colour-blind safe, the repository computes it: Machado (2009) colour-vision-deficiency matrices applied to *linear* RGB — applying them to gamma-encoded values inflates the apparent separation and would let a failing palette pass — with CIE76 colour difference in Lab space and WCAG relative luminance checked at 3:1 for graphic objects. The palette is authored in OKLCH per theme, and a test asserts the resulting bounds.

That analysis produced a counter-intuitive rule now enforced in the code: **the allocation chart must never sort its slices by value.** Sorting makes colour adjacency depend on the data, and re-sorting drops the worst adjacent-protanopia colour difference from 9.1 to 3.2 — below the threshold at which two neighbouring slices remain distinguishable. Fixed group order plus a legend is not decoration; it is load-bearing.

Import parsing

Bank statement import is one auto-detecting engine rather than a parser per bank, because a parser per bank is a maintenance obligation that grows without bound. Column aliases plus a header-row scan cover the major Indian banks; the delimiter is sniffed by *column-count consistency* rather than raw character frequency, because a narration full of commas will otherwise beat a real tab. Both separate debit/credit columns and single signed amounts with a direction indicator are handled, and a default direction is applied only when the file states none at all. XLSX support loads its parser lazily, since it is a 400 KB dependency most users never trigger.

Section 13

Cross-platform parity

The hardest problem in this project was not any single algorithm. It was shipping the same product twice, in two languages, without the two copies quietly drifting apart.

The naive answers both fail. A shared package means one language, which means Flutter-on-web and the compromises that come with it. A code generator means maintaining a code generator. What worked instead was to make disagreement *fail a test*, using three mechanisms.

1 · Byte-identical fixtures

The financial-health test cases exist as a single JSON file duplicated byte for byte on both sides and loaded by both suites. Neither implementation can be adjusted to make its own tests pass without the other's tests failing on the same data. The same pattern covers the notification scheduler.

2 · Cross-language assertions on shared inputs

The Safety Net score is asserted at exactly 100, 25, 50 and 85 for four shared fixtures — in `test/unit/safety_net_test.dart` and in `webapp/src/domain/safetyNet.test.ts`. The same is true of the portfolio roll-up reconciliation invariant and the narrative generator's shared banned-phrase list, which enforces "describe, never advise" identically in both languages.

A five-way version-consistency check runs first in CI, because the version number is a compatibility gate in the backup header — and it had already drifted once, with the app at 1.0.0 while the web and desktop builds said 0.1.0.

Where parity is deliberately not attempted

The Flutter client is the lot-level ledger; the web client still stores aggregated positions and therefore shows unrealised profit and loss only. That is a known and stated gap rather than a hidden one. Deciding *where* parity stops is as much a part of the strategy as enforcing it where it holds — chasing full parity on a feature only one platform's users need would have cost the release.

Section 14

Decisions and trade-offs

The decisions worth defending, each with the alternative that was rejected. These are the questions an interviewer should be asking.

DECISION	REJECTED ALTERNATIVE	REASONING
No server, at all	A thin sync or licence-validation backend	A server implies accounts, a database of other people's finances, a breach to disclose and a bill that outlives interest in the project. Removing it shrank the threat model to one sentence.
Money as arbitrary-precision decimal in TEXT columns	Integer minor units, or floating point	Floating point is disqualified outright. Minor units work until you divide — for interest accrual, allocation and tax, decimal keeps precision where integers force premature rounding. The cost, accepted knowingly, is that aggregation happens in application code.
No key at rest by default	Keeping the derived key in the OS keychain for convenience	Re-deriving from the PIN on every unlock costs about 45 ms and removes an entire class of at-rest compromise. Biometric convenience became opt-in rather than default.
600,000 PBKDF2 iterations	A cheaper KDF for a snappier unlock	The cost is the point — it is the per-guess price an offline attacker pays against an exported backup. 45 ms on web is the unlock latency budget, spent deliberately.
Three storage contracts frozen forever	Renaming them during the rebrand for tidiness	The verifier token, the IndexedDB database name and the backup magic bytes are compared against existing user data. Renaming any of them locks every existing vault out permanently with no recovery. They keep their old names, and the source says why.
Bisection for XIRR	Newton–Raphson	Newton converges faster but can diverge on the irregular sign-changing series real portfolios produce. A slow correct answer beats a fast wrong one, and returning null beats both when the series does not bracket a root.
Untracked metrics leave the denominator	Scoring untracked areas as zero	Scoring absence as failure teaches users to distrust the score and stop looking at it. Withholding the grade below fifty points of tracked weight is more honest than grading a quarter of the picture.
Fixed slice order in allocation charts	Sorting slices by value, as is conventional	Sorting makes colour adjacency data-dependent; the measured worst-case protanopia separation drops from 9.1 to 3.2 and neighbouring slices stop being distinguishable. Measured, not assumed.

DECISION	REJECTED ALTERNATIVE	REASONING
Unknown asset type throws	Falling back to a default type	The old fallback silently mis-taxed anything unrecognised. A tolerant variant exists for the places that genuinely need one; the strict version is the default because wrong tax is worse than an error.
Three data files left empty on purpose	Filling them with plausible values so features light up	Mutual-fund overlap needs real asset-management-company disclosures and index comparison needs real historical closes. Inventing either produces confident, wrong numbers — the worst possible output for a finance tool.
Captured bank alerts never auto-commit	Writing parsed transactions straight to the ledger	A parser that is 95% right silently corrupts a ledger. Captures queue in a review inbox, and the raw message is discarded the instant it is parsed.
No charting library	fl_chart plus Recharts	Two libraries with different rendering models cannot be made to agree pixel for pixel, and parity was the whole strategy. Six painters and nine SVG components is more code and less risk.

Comments as an incident log

A pattern I adopted deliberately and would carry to any team: where a piece of code exists to prevent a specific defect, the comment names that defect, including its error code. The keychain configuration names `errSecMissingEntitlement (-34018)`. The SQLCipher executor names the system-SQLite shadowing pitfall that produces a silently plaintext database. The service worker names the bug where returning `undefined` from a fetch handler rendered the browser's error page instead of the app. The icon check in the release pipeline names App Store rejection `ITMS-90717`.

The value is that these comments survive me. Six months later the reason a line looks strange is written next to it, and nobody "cleans it up".

Section 15

Testing, CI and measured performance

1,633	161	3	2
TEST CASES	TEST FILES	CI PIPELINES	COMMITTED BENCHMARKS

SUITE	FILES	CASES	NOTES
Dart — unit	47		Domain services, pure and fast
Dart — integration	9		Repository and database paths
Dart — widget	7		Screen behaviour
Dart — golden	3	591	Pixel baselines; excluded from CI as host-font dependent
Dart — on-device	3		Encryption round-trip, macOS keychain, notification delivery
Dart — benchmark	1		Prints real numbers, asserts only loose bounds
TypeScript — Vitest	79	1,042	Domain, library, component and route-level tests
Playwright — visual	1	40	Ten routes × desktop and phone × light and dark; frozen clock, Linux baselines from CI

The four test types worth talking about

Invariant tests check that the functions agree with *each other*, not that each one is individually correct. The integrity suite states the problem it exists for: every screen can be individually right and still disagree with the one next to it. Its fixtures are built by the application's own posting functions rather than written by hand — a fixture with hand-written postings can be made to balance by construction and would prove nothing.

Write-path tests cover the mutations rather than the reads. One of them exists because a transfer written over an existing transaction had to retire that transaction, or a single identifier would end up owning two records sharing one pair of ledger legs.

Visual-regression screenshots cover what unit tests cannot see: ten routes in four projects, with the clock frozen, UUIDs replaced by a counter and the network blocked, so a pixel diff means the screen changed and nothing else. A feature that changes a screen deletes its baselines in the same commit; CI then renders new ones on Linux, fails on purpose, and hands them back as a build artefact to be reviewed and committed — so no screenshot ever becomes the reference without a person looking at it.

A critical-path test walks the whole seam: CSV in, parse, valuation, reconciliation, backup, restore into a *second* vault. It was added after a day of real use produced a run of integration bugs while every unit test stayed green — the units were fine, the seams between them were never exercised.

Continuous integration

PIPELINE	WHAT IT DOES
ci.yml	On every push to any branch and every pull request, four jobs. **versions** runs first and cheaply: five files declare the product version and must agree. **webapp** runs `npm ci`, an icon check, Vitest and the static export as hard gates, with lint advisory. **visual** builds the export, installs Chromium and runs the Playwright screenshots against committed Linux baselines. **flutter** runs code generation, `flutter analyze` and the test suite, excluding host-dependent goldens.
release.yml	On a version tag. A four-way desktop matrix — Windows installer and MSI, macOS Apple Silicon and Intel built separately rather than as a universal binary, Linux AppImage and deb pinned to an older Ubuntu so the result runs on current distributions. Opens a draft release.
release-mobile.yml	On a version tag. Asserts the tag matches the manifest *before* a twenty-minute build. Restores the signing keystore from secrets, writes it to disk only for the job's life and removes it unconditionally. Then unzips the built bundle and asserts against the compiled manifest that internet permission is present, backup is disabled, and the SMS permissions are absent.

That last check is a regression test for a bug that shipped once: minification and manifest merging are exactly where a permission or a missing capability quietly reappears, and neither of them fails the build on its own.

Hosting and deployment

The web client is a **Next.js static export** — `output: "export"` with trailing slashes, so every route ships as `<route>/index.html` and there is nothing to run on a server. That is the no-backend constraint applied to hosting: the host serves files, and it could be any CDN. It is on Netlify at **khazana-app.netlify.app**, and the marketing site with the public cut of this case study is a second static folder, `site/`, with its own stricter policy.

NETLIFY.TOML RULE	WHY IT IS THERE
Content-Security-Policy	The page holds a decrypted vault in memory, so CSP is the control that matters. `connect-src 'self'` is the load-bearing line: the client fetches nothing, so a script trying to send a vault to another origin is blocked outright. `'unsafe-inline'` for scripts is a recorded compromise — the no-flash theme script must be inline, and a static host cannot mint per-response nonces. Hashing that script into the header is the follow-up.
HSTS, two years	Web Crypto only exists in a secure context, so a downgrade to http does not weaken the app — it stops it opening.
`no-cache` on sw.js and build.json	A stale service-worker script can pin an old shell across deploys; a stale build id would hide a new version from the update banner.
Immutable `_next/static`	Content-hashed, so cached for a year safely.
Manifest content type	Netlify does not know `.webmanifest` and serves it as a binary type that browsers reject, which silently removes the install prompt.
Permissions-Policy	No camera, microphone, location or payment — stated at the header level, not only in a privacy policy.

Deployment is a deliberate manual step, not continuous: `npm run deploy` from `webapp/` stamps a build id, builds the export and pushes `out/` with the Netlify CLI. CI is the gate; a person decides when a green build reaches users. Two lessons are written into that process. It must run from `webapp/`, because the CLI resolves `netlify.toml` relative to where it is invoked, and deploying from the repository root silently drops every header above. And a manual deploy drifts: in September the live site was found serving a build from three weeks earlier. The update banner covers the other half of that problem — long-lived tabs on an old build — and a deploy job on merge to the trunk branch is the obvious next step.

Releases of the native apps *are* automated, on a version tag: `release.yml` builds the desktop matrix and `release-mobile.yml` builds and signs the Android bundle and checks its compiled manifest, as described above.

Measured, not claimed

The performance numbers below come from committed scripts anyone can re-run, and they are recorded as measurements on a development host rather than as guarantees.

PRIMITIVE	PATH	MEASURED
PBKDF2-HMAC-SHA256, 600k → 256-bit key	Web Crypto	≈ 49 ms
PBKDF2-HMAC-SHA256, 600k → 256-bit key	Pure Dart, JIT	≈ 1.2 s
AES-256-GCM encrypt, transaction-sized record	Web Crypto	≈ 0.025 ms
AES-256-GCM decrypt	Web Crypto	≈ 0.030 ms
FTS5 search across 10,000 transactions	SQLite via Drift	≈ 0.13 ms

The 600,000-iteration cost is intentional; ~49 ms is the unlock latency budget, spent for brute-force resistance. The pure-Dart figure is the just-in-time test-VM path — release builds are ahead-of-time compiled and faster, and Flutter on web uses Web Crypto directly.

Section 16

Free for everyone

Khazana is free for everyone: every feature, on every platform, with no subscription, no paywall, no in-app purchase, no ads and no account.

An earlier version sold a one-time Pro unlock, with offline Ed25519 licence keys for the web and desktop builds and store purchases on mobile. In October 2026 that tier was removed from both clients. There is no entitlement code, no licence verification and no billing library left in either app, and both test suites assert that no screen is gated.

Two rules that outlived the paywall

> NON-NEGOTIABLE
>
> **1. Backup export, restore, plain CSV dump and erase-everything are always available.** Never hold a user's own data hostage.
> **2. The ledger is never capped.** No transaction, account, budget or holding limits — because a capped ledger reports wrong totals, and every derived figure inherits the lie.

Section 17

Metrics, limitations and roadmap

By the numbers

MEASURE	VALUE
Hand-written source and tests	≈ 98,200 lines
Dart — application code (excluding generated)	40,636 lines across 241 files
TypeScript and TSX — web client, with tests	47,117 lines across 253 files
Dart tests	10,487 lines
Generated Drift code (not counted above)	≈ 18,700 lines
Tracked files in the repository	989
Domain services / modules	28 Dart · 33 TypeScript
Encrypted database tables	23 + FTS5 index
Screens and routes	26 Flutter · 31 web
Automated tests	1,633 across 161 files, plus 40 visual baselines
Platforms targeted	iOS, Android, macOS, Web, Windows/Linux desktop
Commits · active days · elapsed	156 · 27 · 17 weeks
Servers, accounts, telemetry endpoints	0

Built from 22 June 2026, single-handed, alongside full-time work — which is why the commit history is twenty-seven dense days rather than a steady drip. The largest single stretch was fifteen commits on 1 August.

What does not work yet

The repository maintains a numbered defect log with status markers, and several entries have been fixed on the current hardening branch. These are the open ones, reproduced rather than laundered:

ITEM	STATUS
Transaction editing OPEN	Editing exists on the web client but not yet as a dedicated Flutter screen.
Desktop content policy OPEN	The Tauri wrapper does not carry the content-security policy the hosted web build enforces.
Web lot-level parity OPEN	The web client stores aggregated positions and therefore shows unrealised profit and loss only.
Android capture path OPEN	The Kotlin notification-listener bridge is unverified on a physical device.
Encryption tests not in CI OPEN	The on-device tests that guard encryption at rest need a real device, so they are in the manual release checklist rather than the pipeline — which means the tests guarding the product's core promise do not run automatically.
Unsigned desktop installers OPEN	Windows SmartScreen and macOS Gatekeeper will warn. Stated in the release notes rather than left for the user to discover.

ITEM	STATUS
A beta dependency OPEN	Secure storage is pinned to a beta release. A beta is not a shipping dependency for a security product; it is a launch blocker on the checklist.
Web lint debt OPEN	Pre-existing rule violations under the newest ruleset. Lint is advisory in CI with the reason recorded; tests and the build are hard gates.
No pagination anywhere OPEN	Defensible at local-first data volumes, but it is the clearest remaining performance gap.
Design-token drift OPEN	Flutter tokens are copied by hand from the web stylesheet. Its success and danger colours are a shade off the web's, one older radius table disagrees with the current one, and the radius checks in `check-tokens.mjs` still expect the pre-Zentra scale and so never fire. The fix is generating the Dart tokens from the CSS.
Four non-atomic writes OPEN	Running due recurring rules, posting one occurrence, undoing an import and deleting a profile still write record by record rather than through the single atomic batch every other write uses.
No cross-tab sync OPEN	Each tab holds its own key and its own copy of the store, so a write in one tab appears in another only after it is unlocked again. Deliberate for now — sharing state across tabs means sharing the key — but a real limitation.
Flutter periods OPEN	The Flutter dashboard's 1M and 3M windows are 30- and 90-day spans; the web client uses calendar months.
One golden test OPEN	Font rendering differs between hosts; goldens pass only on the machine that generated them, so they are excluded from CI.

Closed since the previous revision of this document, each with a test that would catch its return: the dashboard's net-worth card summed transactions while the rest of the product read the ledger, so opening balances were missing from its cash and net worth — it now uses the same function, pinned by a test with an opening balance in the fixture; receipt attachments are now AES-256-GCM sealed under the vault key and purged with it, and they round-trip through the repository; recurring and imported transactions now write through the ledger writer, pinned by a test asserting that nothing outside the dependency wiring reaches the transaction DAO directly; and the backup restore path is implemented, reachable and round-tripped, including refusing a wrong key, a corrupted file and a backup from a newer build. The first of those was the item this section previously called the clearest violation of the project's own encryption contract.

What comes next, in order

1. Upgrade off the beta secure-storage dependency and verify the Android capture path on hardware.
2. Start the twelve-tester closed track, since fourteen continuous days is the longest lead time in the plan.
3. Code signing for desktop; content-security policy for the Tauri wrapper.
4. Optional peer-to-peer sync between a user's own devices, using the merge function that already exists and is already tested.

Appendix A

LinkedIn copy

Written to be posted as-is. Adjust the closing line to match whether you are hiring, looking, or simply shipping.

SHORT POST — ANNOUNCEMENT

I spent two months building the personal finance app I actually wanted to use.

Khazana keeps every rupee of your financial data encrypted on your own device. No backend. No account. No telemetry. Your PIN derives the encryption key, the key is never stored, and there is no recovery — because there is nobody who could recover it.

Under that vault: double-entry accounting, lot-level portfolio tracking with FIFO matching, XIRR, an Indian capital-gains engine versioned by disposal date, EMI payoff simulation, and a financial-health score.

It ships as two independent clients — Flutter for iOS, Android and macOS, and Next.js for web and desktop — sharing one framework-free financial engine written twice and kept honest by mirrored tests. ~98,200 lines, 1,633 tests, 23 encrypted tables, zero servers.

Writing up the architecture in detail. Happy to talk about any of it.

#Flutter #NextJS #Cryptography #FinTech #SoftwareArchitecture

LONG POST — THE BUILD STORY

Every personal finance app I tried wanted one of two things: my banking credentials, or a copy of my complete financial life on their servers.

So I built the one that asks for neither.

Khazana is offline-first and serverless by architecture, not by configuration. I held one rule for the entire build: no server, ever. Not a light one, not "just for sync", not "just to validate licences". That single refusal forced better engineering everywhere else.

Three things it forced:

→ **Sync without a coordinator.** Two devices reconcile encrypted backups through a deterministic last-write-wins merge with 90-day tombstones. The function is symmetric — same result whichever side runs it.

→ **One source of truth on the client.** Every figure is derived on read from one encrypted store, and every write is one atomic batch followed by one reload — so the dashboard, accounts, budgets and reports cannot disagree about the same rupee.

→ **A threat model I can state in one sentence:** on-device data must be unreadable without the user's PIN. No JWT, OAuth, RBAC, CORS or CSRF surface exists to get wrong, because no endpoint exists.

The hardest problem wasn't cryptography. It was shipping the same product twice — Flutter and Next.js — without the two copies drifting apart on what a number means. The answer was to make disagreement fail a test: byte-identical JSON fixtures loaded by both the Dart and the TypeScript suites, and cross-language assertions pinning the same inputs to the same outputs on both sides.

My favourite decision came out of a bug. I had been storing the derived key in the OS keychain for biometric unlock. On macOS that broke on every rebuild, because keychain items are bound to the code signature that created them. I could have tweaked the configuration. Instead I changed the model: store only the salt and a verifier, re-derive from the PIN each time, make biometrics opt-in. The result is that in the default flow there is now no key at rest at all.

A security improvement that arrived through a usability failure. Most of them do.

~98,200 lines, 1,633 tests, 23 encrypted tables, 5 platforms, 0 servers. Full technical write-up in the comments.

HEADLINE

Software Engineer · Building Khazana, an offline-first encrypted finance app | Flutter · Next.js · Applied Cryptography

ALTERNATIVE HEADLINE

I build software that doesn't need your data on my servers · Flutter · TypeScript · Local-first architecture

ABOUT SECTION

I build products where the architecture reflects a position, not just a requirement. Most recently that is Khazana — an offline-first personal finance app with no backend, no account and no telemetry, where every rupee of data is encrypted on the user's own device behind a PIN that is never stored.

It ships as two independent clients, a Flutter app and a Next.js web and desktop app, over one framework-free financial engine covering double-entry accounting, lot-level portfolio analytics with FIFO matching, XIRR, Indian capital-gains estimation and weighted financial-health scoring. About 98,200 lines and 1,633 automated tests, including cross-language tests that make the two clients provably agree.

I care most about the decisions that are hard to reverse: how money is represented, where the trust boundary sits, and what a system promises when it has no server to fall back on.

Appendix B

Website and portfolio copy

HERO

Khazana

Your money, on your device, encrypted, and nowhere else.

An offline-first personal finance and wealth app with no backend, no account and no telemetry. Double-entry accounting, lot-level portfolio analytics and a real tax engine — behind a PIN that never leaves the device.

FEATURE GRID — SIX TILES

Encrypted at rest. AES-256 on every platform, keyed by PBKDF2-HMAC-SHA256 at 600,000 iterations. The key is derived from your PIN and never stored.

No server, by design. No account to create, no data to breach, nothing that stops working when someone stops paying a hosting bill.

Real accounting. Double-entry postings where every journal entry sums to zero — so net worth is derived, not guessed.

Portfolio that computes. Lot-level FIFO matching, XIRR, sector-wise profit and loss, and capital-gains estimation under rules versioned by disposal date.

Money that is exact. Arbitrary-precision decimal end to end, stored as text. Totals never drift by a rounding cent.

Two apps, one engine. Mobile and web compute identical numbers — enforced by tests that fail if the two implementations ever disagree.

TECHNOLOGIES

Flutter · Dart · Riverpod · go_router · Drift · SQLCipher
Next.js 16 · React 19 · TypeScript · Zustand · Dexie · Tailwind
Tauri 2 · Rust shell
Web Crypto · AES-256-GCM · PBKDF2 · SHA-256
SQLite FTS5 · IndexedDB · decimal arithmetic
GitHub Actions · Vitest · flutter_test · golden tests

AT A GLANCE

≈98,200 lines hand-written
1,633 automated tests
23 encrypted tables, 6 schema migrations
27 + 30 pure domain modules
5 platforms
0 servers
Solo build, 12 weeks

PROJECT BLURB — 150 WORDS

Khazana is an offline-first personal finance and wealth application built on a single constraint: no server, ever. Every rupee of data is encrypted at rest on the user's own device — SQLCipher on mobile, per-record AES-256-GCM on web — behind a key derived from a PIN that is never stored and has no recovery path.

It ships as two independently implemented clients, a Flutter app for iOS, Android and macOS and a Next.js app for web and desktop, sharing one framework-free financial engine: double-entry accounting, lot-level portfolio tracking with FIFO matching, XIRR, an Indian capital-gains engine versioned by disposal date, EMI payoff simulation and weighted financial-health scoring.

Keeping two implementations honest was the central engineering problem. The answer was to make disagreement fail a test — byte-identical fixtures and cross-language assertions across roughly 98,200 lines and 1,633 tests.

ONE-LINE VERSIONS

An encrypted, serverless personal finance app — two clients, one financial engine, zero backends.

Personal finance software that cannot leak your data, because it never has it.

Local-first finance: AES-256 at rest, double-entry accounting, lot-level portfolio analytics, no account required.

Appendix C

Interview question bank

Sixty-three questions this project invites, with answers already thought through. Answer in your own words — the point is that the reasoning is ready, not the phrasing.

Architecture

Why no backend? Isn't that just avoiding the hard part?

It moves the hard part rather than avoiding it. Removing the server removed an entire threat surface — no JWT, OAuth, RBAC, CORS or CSRF to get wrong — but it made three things genuinely harder: multi-device reconciliation with no coordinator, keeping every screen consistent with no server to be the source of truth, and migrations that run on a database I can never inspect. I solved those with a symmetric last-write-wins merge with tombstones, one encrypted store that every screen derives from, and idempotent migrations. If the role needs distributed systems experience I would talk about different work — but I would not describe this as the easy path.

Why two codebases instead of Flutter Web?

Flutter Web renders to canvas, which is heavy to load and awkward to make properly accessible. I wanted the web client to be real DOM, statically exported, installable as a PWA and wrappable in Tauri. The cost is duplicated UI. The benefit I did not anticipate is that having to write the financial logic twice forced it out of the UI layer completely — which turned out to be the best structural decision in the project.

Walk me through what happens when a user opens the app.

Cold start shows a lock gate, never data. The router's redirect sends every application route to the unlock screen unless the vault state machine says unlocked. The user enters a PIN; it is stretched with PBKDF2-HMAC-SHA256 at 600,000 iterations against a stored 32-byte salt — off the UI isolate on native so it does not jank. The PIN was never stored, so correctness is checked by decrypting a verifier token encrypted at setup; AES-GCM is authenticated, so a wrong key fails the tag and throws. On success the key is held only in memory, records are read and decrypted, filtered to the active profile, and handed to the store. On lock the key and all decrypted data are cleared.

Data and correctness

How do you handle money, and why not integer paise?

Arbitrary-precision decimal end to end, stored as text, never floating point. Integer minor units are a perfectly good answer and I considered them — they fail for me at division. Interest accrual, allocation percentages and tax computation all divide, and integers force you to round early and repeatedly. Decimal lets me carry precision to the last step and round once, deliberately, where the rule says to. The cost I accepted is that aggregation and sorting happen in application code rather than in SQL, since the column is text.

How do you know your numbers are right?

Three layers. Unit tests for each pure function. Invariant tests that check the functions agree with each other — a roll-up along any dimension must sum exactly to the portfolio total, and assets plus liabilities plus equity must be identically zero. And a critical-path test that walks CSV import through valuation, reconciliation, backup and restore into a second vault. I added that third layer after a day of real use produced a run of integration bugs while every unit test stayed green; the units were fine and the seams between them had never been exercised.

Explain XIRR and why you implemented it the way you did.

XIRR is the discount rate at which the net present value of an irregular series of cash flows is zero — money-weighted return, so it accounts for when you invested, not just how much. I find it by bisection over a rate range of about −99.99% to 1000%, 200 iterations, tolerance 1e-7. Newton–Raphson converges faster but can diverge on the irregular sign-changing series a real portfolio produces. And when the series has no sign change, or fewer than two flows, I return null rather than a number — a fabricated return is worse than no return.

You have two implementations of the same logic. How do they not drift?

By making drift a test failure. The health-score cases are one JSON file duplicated byte for byte and loaded by both suites, so neither side can be tuned to pass without breaking the other. The Safety Net score is asserted at the same four values in both languages on the same fixtures. There is also a version-consistency check that runs first in CI across five declaration sites, because the version is a compatibility gate in the backup header and it had already drifted once.

Security

How do you know it's actually secure? You're not a security engineer.

I would not claim it is audited. What I would claim is that the model is small enough to state precisely and that each control is written down with the threat it addresses. There is no server, so the whole question is whether on-device data is unreadable without the PIN. Standard primitives, used conventionally: PBKDF2 at 600,000 iterations, AES-256-GCM with a fresh IV per record, SQLCipher for whole-file encryption. The parts I would want a reviewer to look at hardest are the backup envelope validation and the lockout logic. And I document what it does not defend against — a compromised device, a forgotten PIN, coercion — because a security claim without a stated boundary is marketing.

Why 600,000 iterations? Users hate slow unlocks.

The cost is the product, not a side effect. It sets the per-guess price for an attacker running offline against an exported backup file. About 49 ms on web is a latency I am willing to spend for that, and it is measured rather than assumed — there is a committed benchmark. I also enforce a minimum PIN length at setup and an exponential lockout, and the lockout is checked *before* the derivation runs so an attacker cannot use the rate limiter to burn our CPU.

No password recovery at all? That's a support nightmare.

It is, and it is the correct trade-off for this product. Recovery requires either an escrowed key or a server that can reset something — and both re-introduce the exact trust relationship the app exists to avoid. What I owe the user instead is clarity: it is stated at setup, and backup export is free forever precisely so that the user's own encrypted copy is always their recovery path. The availability risk is real and I name it in the threat model rather than discovering it in reviews.

Judgement

What was the hardest bug?

The macOS keychain failure — see the STAR story below. The honourable mention is subtler: the web client's base64 encoder used spread-argument character conversion, which throws a range error past roughly a hundred thousand elements. It worked in every test and failed on the one thing that matters — a real, full-sized vault backup. The lesson I took is that the code path handling the largest input is often the one no test ever gives a large input to.

What would you do differently?

Three things. First, I would build the double-entry ledger from day one instead of migrating to it at schema version three — the migration that converts every existing transaction into balanced postings was avoidable work created by my own earlier shortcut. Same story for lot-level holdings at version four: the original aggregated model made profit and loss literally uncomputable. Second, I would put the cross-language fixtures in place before writing the second implementation rather than after. Third, I would set up CI before feature work, not on the same branch as it.

This is a solo project. How do I know you can work in a team?

Fair question, and I would point at the artefacts rather than argue. I wrote an architecture document, a threat model, and a screen-by-screen inventory that includes a numbered defect log with honest status markers — including entries that say the product is currently wrong. I enforce decisions in CI rather than by memory. And comments in the codebase name the specific defect they exist to prevent, with error codes, so the reason a strange line exists survives me. Those are all habits for other people's benefit; I just happened to be the only other person.

Why should I believe your test numbers?

Count them: the test files and cases are in the repository and greppable, and the performance figures come from committed benchmark scripts you can re-run rather than numbers I typed into a document. That distinction is deliberate — the README says the numbers are measured, not asserted from a specification, and records that they are host-dependent.

What's the weakest part of the codebase?

Until recently it was attachments — the one place the encryption contract was not honoured, and I treated it as the priority precisely because it contradicted the product's central claim rather than because anyone had complained. Receipts are now sealed with AES-256-GCM under the same key as the database, the plaintext never touches disk even briefly, and a file that fails authentication raises instead of quietly rendering blank, because silent failure would hide tampering. What I would point at now is that there is no pagination anywhere: defensible at local-first data volumes, but it is the clearest remaining performance gap, and it is the kind of thing that is cheap to fix early and expensive once someone has a decade of transactions.

How did you decide what not to build?

Mostly by refusing to fake data. Three data files in the repository are deliberately empty: mutual-fund overlap analysis needs real asset-management-company disclosures, and index comparison needs real historical closes. I could have populated both with plausible values and shipped features that look complete. For a finance tool, confident wrong numbers are the worst possible output — worse than a missing feature, because the user cannot tell. Same reasoning for statement parsing of password-protected consolidated account statements, and for broker API sync, which would send the user's portfolio to a third party.

Design levels and UML

Draw me the architecture on a whiteboard.

I would draw it at three magnifications and say which one I am on, because the interesting thing changes at each. Context: a person, the app, and three externals — the OS keychain, one allowlisted quote host that is opt-in and desktop-only, and the app stores. That is the whole boundary. Container: five deployables — the Flutter app, the Next.js app, the Tauri shell, and the two databases — and the thing to notice is that there is no arrow *between* the two clients. They exchange an encrypted file the user carries, which is exactly why the merge had to be a pure symmetric function instead of a protocol. Component: four rings per client with dependencies pointing strictly inward, and the domain ring importing no framework, doing no I/O and taking no clock.

What is the difference between your high-level and low-level design?

The high-level design is the decomposition and the reasoning: one constraint — no server, ever — and then, for each thing a server normally does, what replaced it and what that cost. Authentication became a PIN-derived key and the cost is no recovery. Sync became last-write-wins over exported backups with 90-day tombstones and the cost is that it is not real time. Licensing became offline Ed25519 and the cost is that a leaked key cannot be revoked. The low-level design is four contracts precise enough to implement against: the decimal-to-text converter, the all-or-nothing mutation batch, the idempotent migration ladder, and the backup envelope's byte layout.

Why does your context diagram have almost nothing in it?

Because the emptiness is the product rather than an omission. A typical finance app's context diagram has an aggregator, an identity provider, a database cluster and an analytics pipeline, and every one of those is a place the user's financial history exists outside their control. Mine has the operating system's keychain, a distribution channel, and one optional price feed that receives ticker symbols and nothing else. The web build's content-security policy sets `connect-src` to `'self'`, so there is not merely no exfiltration — there is no origin the page is permitted to reach.

Which UML diagram tells you the most about this system?

The entity-relationship one, because of a single band across the middle of it: postings sharing an entry identifier sum to exactly zero. Everything above that line is structure and everything below it is the portfolio model, but that invariant is the thing the design exists to protect. It is asserted in the integrity test suite and checked again at runtime by the diagnostics screen — the same claim enforced twice, at build time and on real data, because a schema cannot express it.

Where is the dependency inversion, concretely?

The domain ring declares repository interfaces — `TransactionRepository` and twelve others — and the data ring implements them with Drift-backed classes. So the arrow points from infrastructure up to policy. The practical payoff is not architectural purity: it is that the financial-health scorer depends on the shape of an account rather than on Drift, which is precisely why I could port that layer to TypeScript by hand and why it tests in microseconds without a database.

Your state machine has no "logged in" state. Why not?

Because there is no session to be in. There is a key in memory or there is not. Every path out of Unlocked purges it — five minutes idle, two minutes hidden, the app backgrounding, or the lock shortcut — and the only path back in re-derives it from the PIN. A session would be a token that outlives the secret that created it, and that is the thing I did not want to exist.

Screens and product decisions

Why does the diagnostics screen not fix anything?

Because a diagnostic that repairs silently destroys the evidence of how the corruption happened, and then the second occurrence is unexplainable. It reports eleven checks with the offending record identifiers and links each failure to the screen that fixes it. The related decision is that restore is gated on an explicit allowlist of four structural checks rather than on severity — a restore blocked because an exchange rate had not been entered yet is an intact file the user cannot get back into, which is a worse outcome than the warning it was protecting them from.

You have 59 routes. Isn't that too many?

It is a lot of surface, and the honest answer is that the navigation needed two rounds of work because of it. The web client groups routes into five named sections declared once and consumed by the sidebar, the drawer and the command palette. The Flutter client had a real bug here: with five tabs and twenty-seven screens, most locations are not a tab, and a `startsWith` scan that fell back to index zero meant eleven screens all highlighted Dashboard. The fix was an explicit map, because "Reports lives under Score" is a product decision that no string prefix can infer.

How do you stop the two clients drifting apart on navigation?

Tests. One asserts that the web bottom navigation lists the same five destinations as the Flutter tab bar. Navigation is what users build muscle memory for, and two clients disagreeing about where Budget lives are two products wearing one name.

Why is a transfer a separate entity rather than two transactions?

Because moving money between your own accounts is not spending, and the type system should say so rather than every consumer remembering to filter it out. When it was modelled as a transaction it inflated every budget, every expense total and the savings rate. The cost of the fix is that every timeline now merges two entity types instead of querying one — paid once in the query layer, rather than repeatedly and invisibly in every aggregate.

Your tax screen refuses to show some holdings. Why is that good?

Two refusals, both deliberate. Holdings with no recorded purchase date are excluded rather than defaulted to short-term, because defaulting invents the fact that determines the rate. Slab-rate instruments return null with a banner rather than a guess. And the gains are priced as one portfolio rather than per position, because the long-term exemption is an annual allowance applied once across all disposals — computing it per holding would multiply the allowance by the number of positions the user happens to hold. A tax figure that is confidently wrong is worse than one visibly incomplete, because only the second gets corrected before it is filed.

Give me an example of choosing honesty over polish.

The dashboard's net-worth chart prefers recorded snapshots, but a new vault has none, so it falls back to a series derived by walking the transaction history — and labels that line *estimated*. Two code paths for one chart, and a label most users will never read. The alternative is presenting derived history as recorded history, which is the kind of small dishonesty that makes every other number on the screen suspect the moment it is discovered. The portfolio overview does the same thing three times over: banners for unpriced positions, for exchange rates still at their defaults, and for a filtered or past-dated view.

Why does the holdings grid show different columns for different rows?

A fixed deposit has no market price. Rendering one would be an invention, so those rows show rate, accrued interest and maturity instead. The cost is real: the grid has two row shapes and cannot sort meaningfully on a single current-price column across both. Sorting is what I gave up, and the alternative was fabricating a number in a financial application.

What does the forecast do when it does not know?

It returns null with a typed reason — no income history, or no spending history — and the screen explains rather than prints a figure. Safe-to-spend is the minimum of two independent ceilings: cash after committed outgoings, and remaining budget headroom. A number ignoring either would tell someone to spend money already owed, or money they do not have. The screen shows its arithmetic line by line for the same reason: a forecast nobody can audit is a horoscope.

Why is the import flow three steps instead of one?

Parse, preview, confirm — and then the whole run is recorded as a batch that can be undone as a unit. Import is the highest-risk write in the product: it is the one operation that can put hundreds of wrong rows into a ledger in a single click. Skipped rows are counted and shown rather than dropped silently. The undo is honest about its limits too: it removes what the import added, and it says it cannot restore prior values it never captured, rather than implying a general-purpose time machine.

Why does settings ask for two different passphrases?

Export and restore take a passphrase separate from the vault PIN, so a backup can be handed to someone — or stored somewhere less trusted — without handing over the key to the running app. It is one more thing to remember, which is a genuine cost. The related detail on that screen is storage durability: it is surfaced as a row with a retry button rather than swallowed, because Safari evicts script-writable storage after roughly seven days of no visits, and a finance app silently losing its database is not a failure the user should discover on their own.

Data model and migrations

You are on schema version 6. Talk me through the hard migration.

Version 3, which converts every single-entry transaction into a balanced pair of postings and seeds a chart of accounts. Two things made it safe. It uses deterministic identifiers — `acct-cash-$vault`, `acct-exp-$category` — so re-running it cannot create a second cash account, which matters enormously when the database is on a device I can never inspect and cannot roll back. And it preserves net worth exactly, which is the property I asserted rather than the row counts.

How do you migrate safely with no way to inspect production data?

Idempotent by construction, additive where possible, and nullable by default. The health-score columns added at version 5 specifically refuse to back-date invented history — a vault with no score for last March shows nothing there, not zero. And where a migration has to match on something fuzzy, I choose the direction of the safe failure: `reclassifyInvestments` matches on category name, because a false negative leaves the old behaviour in place while a false positive would silently reclassify genuine spending.

Why are prices a table rather than a column on the holding?

Because three features need history and a column can serve none of them: valuation "as on" a past date, day-change calculation, and staleness disclosure. The same reasoning put brokerage, securities transaction tax, stamp duty and GST as columns on the trade rather than folding them into the price — they belong to the acquisition, and cost basis is simply wrong without them, which means the realised gain and therefore the tax are wrong too.

Why can't you use SUM() in SQL?

Because money columns are `TEXT`, so `SUM()` would aggregate lexicographically. That is the consequence of choosing decimal over float, and I followed it through rather than forgetting it: no query in the codebase uses `SUM()` on a money column, the DAOs return rows, and totals are computed in Dart over `Decimal` or in TypeScript over `decimal.js`. It is a real cost in query efficiency, accepted for exactness at the scale a personal ledger actually reaches.

How do you guarantee a transaction and its ledger legs stay consistent?

They are applied as one batch — all the mutations or none. They were once three sequential writes, and a failure between them left an entry with a single leg, which is an unbalanced ledger and the exact state double-entry exists to prevent. The subtle part is that encryption happens deliberately *outside* the database transaction: awaiting a non-Dexie promise inside a Dexie transaction drops out of its zone and commits early, so the obvious fix would have looked correct and silently defeated itself.

What happens when a record fails to decrypt?

It is collected and reported rather than skipped. It used to be skipped without a word, which meant the record vanished from every screen *and* from the next backup export — turning a recoverable problem into permanent loss. Export now refuses outright if any record fails to decrypt, because exporting a partially readable vault is how you make the loss final.

Testing and parity

How do two separate implementations stay in agreement?

Two mechanisms, in increasing order of strength. Byte-identical fixture files loaded by both suites. And cross-language assertions on shared inputs — the same vault produces the same health score in Dart and TypeScript. Where parity is deliberately not attempted, I say so: the web client does not yet have the lot-level model, and that is in the defect log rather than glossed over.

What is an invariant test and why do you need them?

It checks that functions agree with *each other* rather than that each is individually correct — every screen can be right on its own and still contradict the one beside it. A roll-up along any dimension must sum exactly to the portfolio total; assets minus liabilities minus equity must be identically zero. The detail that makes them worth anything is that the fixtures are built by the application's own posting functions, not written by hand: a hand-written fixture can be made to balance by construction and would prove nothing.

Your CI runs lint as advisory. Isn't that a cop-out?

It is debt, and it is recorded as debt with the reason attached rather than hidden by deleting the rule. Tests and the static build are hard gates; lint is advisory because of pre-existing violations under Next 16's newer ruleset. What I would not accept is a green pipeline that is green because the check was removed. The version-consistency job in the same pipeline is the opposite case — it exists because five files each declare the product version, they drifted once, and nothing noticed.

What is not covered by tests?

The encryption-at-rest tests need a real device, so they sit in a manual release checklist rather than the pipeline — which means the tests guarding the product's central promise do not run automatically, and I would rather say that than leave it to be discovered. Golden tests are excluded from CI because font rendering differs between hosts. There is no browser-level functional end-to-end suite — Playwright runs visual regression only — and a critical-path integration test that walks CSV import through valuation, reconciliation, backup and restore into a second vault covers that ground instead.

Product and delivery

Does anyone actually use it?

Ten people are using it now — friends I handed it to — and the Play Store closed-testing track is next, which needs twelve testers for fourteen continuous days. So: real users, not yet a public release, and I would not describe it as more than that. What the small number does buy is the kind of feedback that unit tests cannot produce. The critical-path test exists because a single day of real use produced a run of integration bugs while every unit test stayed green.

How do you make money from it?

I don't. Khazana is free for everyone — no subscription, no paywall, no in-app purchase and no ads. An earlier version sold a one-time Pro unlock verified offline with Ed25519 signatures; it was removed in October 2026. Two rules survived it: the ledger is never limited, and backup, restore, CSV export and erase-all-data are always available.

You used an AI coding assistant. What did you actually do?

I owned the architecture, the security model, the data model and the domain logic, and I reviewed every change before it shipped. The decisions in this document are the evidence: choosing decimal over integer minor units and then following the consequence through to banning `SUM()`; discovering that sector profit and loss was uncomputable under the aggregated model and designing the lot-level replacement; deciding that the default path should keep no encryption key at rest anywhere. An assistant types faster than I do. It does not notice that a feature request is impossible given the schema, and it does not decide what the product refuses to do.

What would you build next, and why that?

In order: get off the beta secure-storage dependency and verify the Android capture path on real hardware, because a beta is not a shipping dependency for a security product. Then start the closed-test track, since fourteen continuous days is the longest lead time in the plan and nothing else is blocked by it. Then code signing and a content-security policy for the desktop wrapper. Peer-to-peer sync between a user's own devices is last despite being the most interesting, because the merge function it needs already exists and is already tested — it is the smallest remaining risk, so it goes after the things that are not.

Front end, theme and data flow

Why CSS variables for the theme rather than a theme object in JavaScript?

Because the browser already has a theming engine and it is faster than mine. Switching theme is one attribute change on `<html>`; every surface repaints from the cascade with no React re-render. A JavaScript theme object would put every component on the render path of a colour change. The variables are also the one format three consumers can read: Tailwind through `@theme inline`, the design-spec build through a parser, and the SVG charts directly, as `var(--series-1)` fills that repaint with the theme. The rule I hold on top of it is that only colour varies between themes — radius, spacing and type are in a separate invariant block — so toggling dark mode can never move a layout.

How do you stop dark mode flashing white on load?

A small inline script in the document head sets `data-theme` before the first paint, reading the saved choice or the system preference. It has to be inline and synchronous — anything bundled runs after the first paint, which is exactly the flash. The cost is that the script duplicates the accent table, and duplicated tables drift, so a test extracts the table from `layout.tsx` and asserts it equals the store's. The same pattern covers a one-time accent migration: it exists in both places and is tested in both.

Why hand-build eighteen chart types instead of using a library?

Three reasons, in order of weight. A finance chart's job is often to *refuse* — to show "estimated", leave a gap for an unpriced day, or stop drawing where data stops — and libraries make the happy path easy and the honest path hard. Every chart had to read the same tokens for line width, bar thickness and colour, in both themes, and match a Flutter twin drawn with a custom painter. And the static export ships to a desktop wrapper where bundle size matters. The cost is real: I own every axis, tooltip and accessibility label. I paid it down with shared pieces — one tokens file, one smoothing function, one entrance hook — and every chart carries `role="img"` with a generated label describing its data.

Walk me through what happens when someone adds a transaction.

The Add screen calls `put`. That builds one batch: the transaction stamped with the active profile, its two ledger legs with deterministic ids so an edit overwrites rather than duplicates, and any account it needs. The batch is encrypted record by record, then written in a single IndexedDB transaction — all or nothing. Then the store reloads: decrypt, filter to the profile, one `set`. Dashboard, Accounts, Budget, Reports and Goals are all subscribed with selectors, so each recomputes its figures from the new arrays. Nothing is patched in place, so there is no screen that can be one write behind.

Reloading everything on every write sounds slow. Why do it?

Because correctness is binary and performance is a curve. A full reload makes stale state structurally impossible: there is no cache to invalidate, so no invalidation bug. At personal-ledger scale — thousands of records, AES-GCM decrypt at about thirty microseconds each — it costs tens of milliseconds. I know where it stops scaling: the next step is incremental updates per record type and finer selectors, and I would add it when a measured vault needs it, not before. Bulk paths already batch — an import is one write and one reload, not one per row.

How do you guarantee two screens show the same number?

Three layers. Structurally: figures are derived on read from one store, never stored per screen. By convention: each figure has exactly one function — "cash" is `liquidBalance` everywhere. By test: an integrity suite asserts that functions agree with each other — a balance equals opening balance plus postings, budget spent equals the sum of the category's transactions, a transfer changes no net worth. The convention is the weak layer, and I found it break: the dashboard's net-worth card was summing transactions while the card beside it read the ledger. It now has a test that pins them together, with an opening balance in the fixture, because that is what the two methods disagree about.

Why is the period not a global setting?

Because screens read time differently. A budget is a calendar month; reports compare arbitrary ranges; the net-worth chart wants one week to all time. A global selector would make every screen reinterpret a choice made for another. What is global is the *model*: one `DateRange` type with inclusive local-time bounds and a label that travels with the range, one bucketing function, one financial-year rule. So the selection is local, and the meaning of a period is shared.

Why does the period carry its own label?

Because the caption is a claim about the query. The dashboard once computed "this month" as the trailing thirty days, so on 9 August its figure labelled August included three weeks of July. Nothing crashed and no test failed. Carrying the label inside the range means the text on screen is generated from the same object that selected the data — they cannot drift.

Why is the dashboard layout in local storage and not in the encrypted vault?

Because it is a fact about a screen, not about money. A layout made on a 1440-pixel monitor is wrong on a phone, and the vault travels in backups to other devices. It is keyed per profile, because two people sharing a laptop arrange it differently. And it is treated as untrusted input on read: unknown ids are dropped, sizes clamped to each widget's minimum, and widgets introduced in a later release are slotted in beside their default neighbours — so neither an old layout nor a new release can break the other.

Is drag-to-rearrange accessible?

It was designed to be. The handle is a button that takes arrow keys as well as pointer drags, the size options are a pressed-state group with spoken names — "Small" through "Full width" — and the card contents are made `inert` during editing so a drag cannot activate a link and keyboard focus does not wander into a card's internals. Reflow animates with a spring, which the reduced-motion setting turns off.

Why is ghost mode not just a CSS blur?

A blur hides the number from eyes and leaves it everywhere else: in the DOM, in the accessibility tree that a screen reader announces in a café, in a screenshot's text layer and in a copy-paste. So masking is a property of the money primitive — `MoneyValue hidden` renders bullets — and charts change their accessible labels to say "hidden". The gap I would name is that it is applied per call site, about two dozen of them, and the test covers persistence rather than every screen.

How do you keep the web and Flutter clients looking like one product?

Partly by tests and partly by discipline, and I would say which is which. Tested: navigation tabs, the contrast of muted text and the colourblind separation of chart series, each asserted on both sides. Discipline: twin component kits and twin chart tokens maintained by hand. That second part has drifted — the Flutter success and danger colours are a shade off the web's, and one older radius table disagrees with the current one. The fix is generating the Dart tokens from the CSS, which is on the list.

How do you make screenshot tests deterministic?

By removing every source of variation. The clock is frozen at a fixed instant in India's time zone, `crypto.randomUUID` is replaced by a counter so the sample vault is identical every run, external network is aborted, motion is reduced, service workers are blocked, and navigation goes through the router so the vault stays unlocked. Baselines are generated on Linux by CI, because font rendering differs by host — the same reason the Flutter goldens are excluded. A feature that changes a screen deletes its baselines in the same commit, so the regenerated images are reviewed as part of that change.

What would you change about the front-end architecture?

Three things. Finer store subscriptions and incremental reload, once a measured vault needs them. Move the remaining sequential writes — running due recurring rules, undoing an import, deleting a profile — onto the same atomic batch every other write uses. And generate the Flutter tokens from the CSS so the design system has one source rather than one source and a careful copy.

CI/CD and hosting

Walk me through your CI/CD.

Every push to any branch runs four GitHub Actions jobs: a version-consistency check first, because it is cheap and once caught real drift; the web job — tests and the static build as hard gates, lint advisory with the reason recorded; Playwright visual regression against committed Linux baselines; and the Flutter job — codegen, analyze, tests. The trunk is protected, so everything lands through a pull request with those checks green. Native releases are tag-driven: the desktop matrix and a signed Android bundle whose compiled manifest is asserted after the build. The web deploy is the one manual step, and I would say so.

How is the web app hosted, and why Netlify?

It is a static export — every route pre-rendered to an HTML file — so any static host works; that is the point of having no backend. Netlify gave me headers as configuration in the repository: a content-security policy whose `connect-src 'self'` means the app cannot talk to any other origin, two-year HSTS because Web Crypto requires a secure context, no-cache on the service worker and the build-id file, immutable caching for hashed assets, and a fix for Netlify serving the web manifest with the wrong content type.

Why is the deploy manual? Isn't that a gap?

It is a trade-off with a known cost. Manual means a person chooses when a green build reaches users, which mattered for changes like removing the paywall. The cost showed up once: the live site sat three weeks behind the branch and nothing noticed. Two mitigations came from it — a recorded rule that the deploy runs from the web folder, since running it elsewhere silently drops the security headers, and the update banner so open tabs learn about a new build. The next step is a deploy job on merge to trunk with a Netlify token as a secret, keeping the pull request as the human gate.

Seven STAR stories

1 · A security improvement that came from a usability bug

Situation. Biometric unlock stored the derived encryption key in the macOS keychain. Every rebuild during development failed vault creation with `errSecMissingEntitlement (-34018)`.

Task. Make biometric unlock work reliably across rebuilds and signing configurations.

Action. The root cause was that keychain items are bound to the code signature that created them, so each rebuild made the app a stranger asking for another application's secret. Rather than tune the keychain configuration, I questioned whether the key needed to be there at all. I changed the default path to store only the salt and a domain-separated SHA-256 verifier, re-deriving the key from the PIN on every unlock — about 45 ms — and made biometric key storage explicitly opt-in. I also consolidated five duplicated keychain configurations, all of which were wrong, into one pinned definition.

Result. The bug went away, and the default flow now has no encryption key at rest anywhere. A usability failure produced a materially better security posture, which is the honest way most improvements arrive.

2 · Discovering a feature was built on an impossible data model

Situation. Users wanted sector-wise profit and loss on the portfolio. Holdings were stored as one aggregated row per symbol with a single average cost, no sector, no price date and no charges.

Task. Ship sector P&L — and first, work out whether it was even computable.

Action. It was not. Realised profit and loss requires knowing which specific lots a sale consumed, and an average-cost row has destroyed that information irreversibly. I designed a lot-level model — instruments, trades, prices, dividends — wrote FIFO matching with deterministic tie-breaking, and migrated the schema. Crucially, I wrote the reconciliation invariant first: a roll-up along any dimension must sum exactly to the portfolio total, asserted in both languages. I also made unpriced holdings count at cost while exposing a count, so the interface can disclose that its own figure is understated rather than quietly presenting it as complete.

Result. Sector, industry, market-cap and asset-class roll-ups that reconcile exactly, plus XIRR and realised-gain tax estimation that were impossible under the old model. The lesson I carry from it: when a feature request seems disproportionately hard, check whether the data model can express the answer at all before estimating the work.

3 · A restore that wrote 367 perfectly good records where nothing could see them

Situation. Restoring a backup onto a second device appeared to succeed and produced an empty app.

Task. Find out why intact, correctly decrypted data was invisible.

Action. The restore was writing records verbatim, including their original profile identifiers. The receiving device had its own profile with a different generated identifier, so the reload filter — which scopes records to the active profile — matched nothing. The data was all there and no screen could see it. I rewrote restore to reconcile profiles by name and kind, remap incoming identifiers, re-write local profile rows still referenced after the clear, and apply the entire restore as a single transaction so the vault is either the old book or the new one and never a mixture. I also made restore validate the backup's own contents before writing anything, and made export refuse outright if any record fails to decrypt — because exporting a partially readable vault makes the loss permanent.

Result. Cross-device restore works, and is now covered by tests including one that restores into a genuinely second vault. The wider lesson: "the write succeeded" and "the user can see it" are different claims, and only the second one is the feature.

4 · Every budget in the product was wrong, and nothing was broken

Situation. Buying a mutual fund was recorded as an expense. Nothing crashed, no test failed, and every screen was internally consistent — but a user who invested ₹20,000 in a month saw it deducted from their spending budget, counted in their expense total, and subtracted from their savings rate. The product was telling people that saving money was spending money.

Task. Correct the classification without corrupting vaults that already held months of mis-typed history.

Action. The root cause was a missing distinction in the type system rather than a bug in any function. An expense and an investment are both cash leaving the account, and the model had collapsed those two facts into one. I introduced a third transaction type and — the part that actually fixed it — a separate `CASH_OUT_TYPES` set, so that "money left the account" and "this was spending" became two different questions that consumers answer explicitly. Budgets, expense totals and the savings rate read the second; cash-flow and account balances read the first. Existing vaults were repaired by a migration that matches on category *name*, chosen deliberately for the direction of its failure: a false negative leaves the old behaviour in place and is visible, whereas a false positive would silently reclassify genuine spending as investment and inflate the user's savings rate in the other direction.

Result. Budgets, expense totals and savings rate became correct, and transfers were separated out on the same principle shortly afterwards. The lesson I took is the one I now look for first: the most expensive defects are not the ones that throw. This one produced plausible numbers on every screen, passed every test, and would have been discovered by a user trusting a savings rate that was wrong by the exact amount they had been most responsible about.

5 · Two numbers on one screen disagreed

Situation. While preparing this document I traced how a single transaction reaches every screen, and found that the dashboard's Net worth card computed cash by summing transactions, while the balance hero on the same page — and Accounts, Forecast, Score and the daily snapshot — read the ledger.

Task. Make every "cash" in the product the same figure, and make sure it could not quietly diverge again.

Action. Summing transactions misses opening balances entirely, so anyone who created a bank account with ₹50,000 in it saw ₹0 on one card and ₹50,000 on the next. The month-change figure was worse: it compared a transaction sum against a snapshot taken from the ledger. I switched the card to `liquidBalance`, the function the rest of the product already used, and fixed the second-order effect — the estimated net-worth line still walks transactions, so I added the opening balances as a constant so its last point still lands on the headline. Then I wrote the test with the one input the two methods disagree about: an account with an opening balance and no transactions. I confirmed it failed on the old code before it passed on the new.

Result. One function for cash everywhere, and a regression test that encodes why. The lesson is the same as the misclassified-investment story, one level up: each card was internally consistent, so nothing failed. Consistency across screens is a property you have to test for directly, with fixtures chosen to make the methods disagree.

6 · A chart that stopped two-thirds of the way across in Firefox

Situation. The net-worth line chart drew itself in with an animation. In Chrome and Safari it was correct; in Firefox the line ended about two-thirds of the way across, leaving only the top edge of the area fill beyond it.

Task. Make the chart identical in every engine without dropping the entrance animation.

Action. The draw-in used the standard trick: `pathLength` plus an animated stroke dash. Combined with `non-scaling-stroke` on a viewBox stretched to the card's width, Firefox measures the dash in a different coordinate space from the path, so the dash ran out early. Rather than tune the numbers per browser, I changed the technique: the line now reveals left to right through an animated clip rectangle, which depends only on geometry that every engine computes the same way. The reduced-motion path skips it entirely.

Result. One rendering in every engine, and a visual-regression suite that covers the dashboard in four configurations so the next engine difference shows up as a pixel diff rather than as a user report. The lesson: when a technique depends on how an engine interprets an underspecified corner, change the technique rather than special-casing the engine.

7 · Users who kept seeing a paywall that no longer existed

Situation. After the deploy that made Khazana free, tabs that were already open still showed Pro paywalls. Reloading a local-first finance app locks the vault, so people leave tabs open for days and keep running whichever version they first loaded.

Task. Tell open tabs a new version exists, without ever throwing away an unlocked session.

Action. The build now stamps an id into a small no-cache file and into the bundle itself. Open tabs check it five seconds after load, every thirty minutes and whenever the tab becomes visible. The decision was what to do on a mismatch, and it depends on state: on the PIN screen there is nothing to lose, so the tab reloads itself; once unlocked, it only shows a dismissible banner, because a forced reload drops the key and demands the PIN mid-task. A failed check reads as "up to date" rather than nagging, and the desktop build skips it, since it updates through its installer.

Result. Deploys reach long-lived tabs within half an hour, or instantly at the lock screen, and nobody loses their session to an update. The lesson: in a local-first app the client is the deployment, and "we shipped the fix" is not true until the open tabs have it.

Appendix D

How this would be built again

The order the repository actually followed, and what each phase had to prove before the next one started. Useful for the question "how would you approach building this?" — and for building the next one faster.

> **THE PRINCIPLE UNDERNEATH THE ORDER**
>
> Each phase establishes a guarantee that later phases are allowed to assume. Encryption comes before features because a feature written against a plaintext store encodes the assumption that data is cheap to read. The ledger comes before the screens because screens written against a single-entry model have to be rewritten when it becomes double-entry — which is exactly what happened here, and is the main thing this ordering is designed to avoid repeating.

#	PHASE	WHAT IT MUST PROVE BEFORE MOVING ON
1	**Encrypted storage**	A vault can be created, locked and unlocked; the key is derived from the PIN and never written down; a wrong PIN fails on an authentication tag. *This was the repository's second commit, and it is the right place for it.*
2	**Backup and restore**	The vault round-trips to a file and back, into a *second* vault. Building this early means every later schema change has a working escape hatch, and it makes the format a first-class design artefact instead of an export bolted on at the end.
3	**Encrypted logging**	Failures leave a trace that is itself encrypted and sanitised. Without this, every bug in phases 4–8 is diagnosed by guessing.
4	**The ledger — double-entry from day one**	Every journal entry balances to zero, asserted by a test whose fixtures are built by the application's own posting functions. **This is the phase I would move earliest if I did it again.** Building single-entry first cost a schema-version-3 migration that converted every existing transaction, and the migration was avoidable work created entirely by the shortcut.
5	**The domain layer, framework-free**	Health score, net worth, budgets and recurrence compute as pure functions over plain entities, with time passed in rather than read. Provable by tests that need no database, no widget tree and no browser.
6	**The lot-level portfolio**	Instruments, trades with their charges, and prices as a dated series. Same lesson as phase 4: the aggregated-position model made realised profit and loss literally uncomputable, and finding that out during a feature request is worse than designing for it up front.
7	**The second client**	Both implementations produce identical numbers for identical inputs. **Write the shared fixtures before the second implementation, not after.** Doing it in that order turns parity from an audit into a guard rail.
8	**Screens**	Only now, because every screen is a view over guarantees that already hold. Screens built before the ledger balanced had to be revisited when it did.
9	**Hardening**	Error boundaries, integrity diagnostics, crash capture, storage-durability disclosure, unlock throttling, undo on the riskiest write. This is where the honest-nulls discipline gets applied consistently rather than per-screen.
10	**Release engineering**	Signed builds, permission assertions against the compiled manifest, version consistency enforced across every file that declares it. *Set the pipeline up before feature work, not on the same branch as it.*

The four decisions to make before writing any code

1. **How money is represented.** Decimal or integer minor units — but decide, write it down, and follow the consequences through the query layer. Retrofitting this touches every file.
2. **Whether the ledger is double-entry.** Balances, net worth and every statement derive from this. Changing it later is a migration over all historical data.
3. **Where the trust boundary sits.** Server, no server, or optional server. It determines the authentication model, the sync model, the licensing model and the threat model simultaneously.
4. **What the product refuses to do.** Mine refused to guess: no fabricated prices, no defaulted purchase dates, no invented history. That rule shaped more screens than any feature decision did.

What I would do differently

- Double-entry and lot-level models from day one — phases 4 and 6 above, both of which arrived as migrations because of an earlier shortcut.
- Cross-language fixtures before the second implementation rather than after it.
- CI configured before feature work, on its own branch.
- Pagination designed in at the list level from the start; it is cheap early and expensive once someone has a decade of transactions.

Appendix E

Résumé kit

The same project cut to four different lengths, plus the STAR stories reduced to the two or three lines a résumé can actually carry. Every figure here is counted from the source tree.

LINK LINE — ALWAYS INCLUDE BOTH

Live app: **khazana-app.netlify.app** · Source: **github.com/sandhoshsivanM/FinTech**

The live link matters more than the repository for this project, because the whole app is the link — no signup, no install, no backend to spin up. A reviewer can be looking at working software in about four seconds, which is a materially different experience from cloning a repository.

STACK LINE — ONE LINE, ORDERED BY WHAT A READER IS SCANNING FOR

Flutter · Dart · Riverpod · Drift · SQLCipher · Next.js 16 · React 19 · TypeScript · Zustand · Dexie · Tauri 2 · Rust · Web Crypto (AES-256-GCM, PBKDF2) · Tailwind v4 · Playwright · SQLite FTS5 · Vitest · GitHub Actions

TWO LINES — WHEN THE RÉSUMÉ IS ALREADY FULL

Offline-first encrypted personal finance app for iOS, Android, macOS, web and desktop, built solo. Double-entry ledger, lot-level portfolio with FIFO matching and XIRR, and an Indian capital-gains engine — with no backend, no account and no telemetry by design.

Financial engine implemented twice, in Dart and TypeScript, and held to identical results by cross-language tests. 1,633 automated tests; AES-256 at rest on every platform, keyed by PBKDF2-HMAC-SHA256 at 600,000 iterations.

FOUR BULLETS — THE DEFAULT

· Built an offline-first personal finance app for five platforms with **no backend, no account and no telemetry** — every record AES-256 encrypted on the device, keyed by a PIN stretched through PBKDF2-HMAC-SHA256 at 600,000 iterations and never stored.

· Designed a **double-entry ledger** over 23 encrypted tables with six schema migrations, and a lot-level portfolio model with FIFO matching, XIRR and a capital-gains engine versioned by disposal date. Money is arbitrary-precision decimal throughout; floating point is banned.

· Shipped **two independent clients** — Flutter and Next.js — over one framework-free financial engine written in both languages and proven to agree by cross-language tests on shared fixtures. **1,633 automated tests** across both suites, gated in CI.

· Owned architecture, security model, data model and domain logic end to end, using an AI assistant for implementation and reviewing every change before it shipped. Live at khazana-app.netlify.app; in private beta with ten users, Play Store closed testing next.

SIX BULLETS — FOR A RÉSUMÉ WITH ROOM, OR A PORTFOLIO PAGE

· The four bullets above, plus:

· Implemented a **serverless multi-device merge** — deterministic last-write-wins over encrypted backup files with 90-day tombstones, symmetric by construction so both devices reach the same state — and offline **Ed25519 licence verification**, so purchases work with no activation server.

· Hardened the product against its own failure modes: per-route error boundaries, an eleven-check integrity diagnostic that deliberately never writes, encrypted crash logging with PII sanitisation, durable unlock throttling, and per-run undo on imports. Benchmarks are committed and re-runnable rather than asserted.

STAR STORIES AT RÉSUMÉ SCALE — SAY THE LONG VERSION OUT LOUD, WRITE THE SHORT ONE

Security from a usability bug. Biometric unlock stored the encryption key in the macOS keychain and failed on every rebuild with `errSecMissingEntitlement`. Rather than tune the keychain, questioned whether the key needed to be there — changed the default to store only a salt and a verifier, re-deriving the key per unlock in ~49 ms. *Result: the default flow now keeps no encryption key at rest anywhere.*

A feature that was impossible, not hard. Sector-wise P&L was requested against a schema holding one aggregated row per symbol. Realised gains need to know which lots a sale consumed, and average cost destroys that irreversibly. Designed a lot-level model, wrote the reconciliation invariant first, and migrated. *Result: roll-ups along four dimensions that reconcile exactly, plus XIRR and tax estimation that the old model could not express.*

A restore that wrote 367 invisible records. Cross-device restore appeared to succeed and produced an empty app: records were written with their original profile identifiers, which the active-profile filter did not match. Rewrote restore to reconcile profiles, remap identifiers and apply as one transaction. *Result: working cross-device restore, covered by a test that restores into a genuinely second vault.*

Every budget wrong, nothing broken. Investments were classified as expenses, so investing deflated the user's own savings rate. No test failed — the defect produced plausible numbers everywhere. Separated "cash left the account" from "this was spending" in the type system and migrated existing vaults, choosing a match rule whose failure direction was visible rather than silent. *Result: correct budgets and savings rate; the habit of looking for defects that do not throw.*

TWO THINGS TO KEEP STRAIGHT IN INTERVIEWS

The user numbers are small and stating them plainly is stronger than implying scale: ten people are using it, the Play Store closed track needs twelve testers for fourteen days, and it is not publicly released. Every other figure in this document is countable from the repository, which is the reason to lead with those instead. Second, the AI-assistance bullet stays in. Volunteering it, alongside a clear account of which decisions were yours, is far better than having it inferred — and the architectural decisions in Appendix C are the evidence that the ownership claim is real.

Appendix F

The guided tour — explaining Khazana screen by screen

A five-minute spoken walkthrough for a screen share or a whiteboard. For each stop: what the interviewer sees, how it is built, and the one sentence to say. Load the sample data first, so every screen has something in it.

> **THE SPINE OF THE STORY**
>
> Open with the constraint, then show the screens, then show consistency by adding one transaction and visiting the screens it changes. *"No server, ever — so the browser is the database, the key lives only in memory, and every number on every screen is derived from one encrypted ledger."* Everything below hangs off that sentence.

1 · Vault unlock ≈ 30 s

THEY SEE A PIN screen with the brand mark, trust points and no "forgot PIN" link.

BUILT WITH PBKDF2-SHA256 at 600,000 iterations through Web Crypto, about 49 ms; the key is held in memory only. A wrong PIN fails on the AES-GCM tag of a verifier, not against a stored hash.

> **Say:** "There is no recovery, because there is nobody who could recover it. That is the trade I made for having no server."

2 · The shell and the theme ≈ 30 s

THEY SEE Grey chrome, a near-black active pill, colour only on numbers. Toggle dark mode, change the accent, narrow the window past 900px to show the dock and the raised +.

BUILT WITH CSS custom properties in one file; a theme switch is one attribute on `<html>`. Routes declared once in `navConfig.ts` feed the sidebar, the drawer and ⌘K.

> **Say:** "Ink is interaction, gold is identity, blue is data. Theme changes colour and nothing else, so nothing moves when you toggle it."

3 · Dashboard ≈ 60 s

THEY SEE Balance hero, wallet stack, net worth with an *estimated* label on a new vault, and the chart board. Press Customize: drag a card, make one XL, hide one, then Reset.

BUILT WITH 28 widgets on a 12/6/1-column grid; `packSpans` widens the last card in a short row so no row has a gap; each card is a size container. Eighteen chart types, no library. Layout saved per profile on this device.

> **Say:** "Every card answers one question with the chart form that suits it — and the layout survives new widgets in future releases because a saved layout is treated as untrusted input."

4 · Add one transaction, and watch it ripple ≈ 60 s — the key moment

THEY SEE Add a ₹2,000 grocery expense. Return to the dashboard: cash, this month's spend and the weekly chart moved. Open Accounts: the balance moved. Open Budget: Groceries moved.

BUILT WITH One batch — the entry plus two ledger legs — encrypted, then written in one IndexedDB transaction. Then one store reload and one `set`; every screen recomputes from the same arrays.

> **Say:** "Nothing is stored twice. Balances are opening balance plus postings, computed on read, so there is no cache that can be a write behind."

5 · Budget and time ≈ 30 s

THEY SEE Step back a month with the month navigator; the caption and the figures change together.

BUILT WITH A `DateRange` with inclusive local-time bounds whose label travels with it; calendar months, India's 1 April financial year, one bucketing function for every chart.

> **Say:** "The words on screen come from the same object that selected the data, so 'August' can never contain July."

6 · Investments, Score, Reports ≈ 45 s

THEY SEE Allocation donuts and banners for unpriced positions; the health gauge with its method; a report over a custom range exported to PDF and Excel.

BUILT WITH Pure domain functions, mirrored in Dart and TypeScript with shared fixtures; an export generated in the browser with nothing uploaded.

> **Say:** "When the app does not know, it says so — an unpriced holding is disclosed, not quietly valued at a guess."

7 · Settings, then ghost mode ≈ 45 s

THEY SEE Choose a profile photo, switch profile and see the data and layout change, export an encrypted backup, then press the ghost icon: every amount becomes bullets.

BUILT WITH Photo cropped to 256px JPEG with EXIF stripped, stored in the encrypted vault. Profiles are filtered once in `reload()`. Ghost mode is a property of the money component, not a blur.

> **Say:** "A blur leaves the number in the DOM and in the screen reader. Masking the value is the only version that actually hides it."

The thirty-second whiteboard answer: "How does the data stay consistent?"

One encrypted store, one write path, one reload. Writes are batched — an entry and its two ledger legs commit together or not at all. After every write the store reloads once and every screen re-derives its figures from the same arrays: balances are opening balance plus postings, never a stored total. Each figure has one function, each period one model with its label attached, and an integrity suite asserts the functions agree with each other. Where that convention once slipped, between two dashboard cards, a test now pins them together.

Prepared from the Khazana repository on 16 September 2026; revised 17 October 2026. Figures counted from the source tree; performance numbers reproduced from committed benchmark scripts. Sandhosh Sivan · sandhoshsivan00@gmail.com